

Машинное обучение, ФКН ВШЭ

Практическое домашнее задание. Нейронные сети

Общая информация

Дата выдачи: 24.03.2026

Мягкий дедлайн: 14.04.2026 23:59

Жесткий дедлайн: 19.04.2026 23:59

О задании

В этой работе две независимые части:

- Часть 1. Separable Physics-Informed Neural Network (SPINN) для решения 3D-уравнения Гельмгольца - 4 балла.
- Часть 2. CNN-классификация звуков на датасете UrbanSound8K - 6 баллов.

Оценивание

Максимально допустимая оценка за работу - 10 баллов. 4 балла за SPINN и 6 баллов за CNN.

Задание выполняется самостоятельно. "Похожие" решения считаются плагиатом: все задействованные студенты (в том числе те, у кого списали) получают не более 0 баллов. Если вы использовали внешний источник (даже частично), укажите ссылку в отдельном блоке в конце работы.

Неэффективная реализация кода может негативно отразиться на оценке. Оценка также может быть снижена за плохо читаемый код и плохо оформленные диаграммы/таблицы. Изменять проверочные ячейки и пороги `assert` запрещено.

Устная проверка. Для проверки понимания кода и выводов студент может быть приглашён на устную защиту. Оценка за задание может быть изменена после устной защиты. Если студент не может объяснить ключевые части решения и принятые решения, работа считается недобросовестной и оценивается в 0 баллов независимо от автотестов.

Про вычислительные ресурсы

Задание рассчитано на выполнение в бесплатном Google Colab или Kaggle Notebook, если реализация сделана разумно.

- Используйте GPU экономно: черновую отладку, проверку кода и подготовку пайплайна лучше сначала выполнять на CPU, а GPU подключать уже для обучения и финальных экспериментов. Это поможет избежать лишних проблем с памятью и не тратить ресурсы впустую.
- Для SPINN обучение должно занимать умеренное время; если одна конфигурация обучается слишком долго, стоит проверить размер модели, число коллокационных точек и эффективность реализации.
- Для UrbanSound8K рекомендуется предвычислять спектрограммы и сохранять их на диск. Без этого эксперименты часто упираются в CPU-препроцессинг, а не в обучение модели.
- Для подбора гиперпараметров разумно сначала работать на одном фиксированном dev-split, а полный 10-fold cross-validation запускать только для финальной модели.
- При обучении свёрточных сетей может помочь `torch.backends.cudnn.benchmark = True`.

Формат сдачи

Задания сдаются через систему Anytask. Необходимо прислать ноутбук с выполненным заданием; к ноутбуку прикрепите два файла с весами лучших моделей для каждой из двух задач: `best_spinn_model.pth` и `best_cnn_model.pth`. Сам ноутбук называйте в формате `homework-practice-04-dl-Username.ipynb`, где Username - ваша фамилия.

Для удобства проверки самостоятельно посчитайте свою максимальную оценку, исходя из набора выполненных задач и достигнутого качества, и укажите её ниже.

Оценка: 10

```
In [13]: import torch
import torchaudio
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, Dataset

import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
from sklearn.preprocessing import LabelEncoder

from typing import Tuple, List
from tqdm.auto import tqdm, trange
from IPython import display

import librosa
import librosa.display
from IPython.display import Audio
```

```
In [14]: random.seed(1337)
np.random.seed(1337)
torch.manual_seed(1337)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cpu

Часть 1. SPINN для 3D-уравнения Гельмгольца (4 балла)

Разбалловка

Для удобства разбалловка задания представлена в таблице ниже. В первой части основное внимание уделяется корректной реализации SPINN-архитектуры, организации обучения и анализу результатов.

Подзадача	Баллы	Что оценивается
1.1 Реализация separable-архитектуры	2,0	Реализация класса <code>BaseSPINN</code> : три body-сети, корректная separable-сборка решения на решётке и на произвольных точках, согласованность с <code>BaseModel</code>
1.2 Обучение и качество	1,5	Модель сходится, обучение организовано корректно, результат воспроизводим, достигнуто разумное качество на тестовой сетке

| 1.3 Анализ экспериментов | 0,5 | Краткий разбор выбранной конфигурации и наблюдений по качеству и скорости

Что такое PINN

Physics-Informed Neural Networks (PINNs) - это нейросетевые модели, которые используют для обучения не только данные, но и структуру самой физической задачи.

Идея состоит в том, что модель должна не просто приближать неизвестную функцию, а строить решение, которое согласовано с заданным дифференциальным уравнением и граничными условиями.

В отличие от стандартной постановки supervised learning, где у нас есть множество пар "вход-ответ", в PINN функция потерь обычно включает две части:

- PDE loss - это невязка дифференциального уравнения внутри области;
- BC loss - это ошибка на границе области, где известны граничные условия.

Благодаря этому PINN можно применять даже в ситуациях, когда наблюдений мало, но сама структура задачи хорошо известна.

Физика - ммм, уже испугались?)

На деле, глубоко разбираться в физике здесь не требуется. Вся постановка задачи первой части - уравнение, точное решение, функция источника и граничные условия - уже реализована в `BaseModel`.

Для выполнения задания достаточно понимать следующее:

- есть неизвестная функция $u(x, y, z)$, которую нужно приблизить нейросетью;
- внутри области решение должно удовлетворять дифференциальному уравнению (PDE loss);
- на границе куба решение должно обращаться в ноль (BC loss);
- точное решение нам известно, поэтому мы можем измерять ошибку модели после обучения.

Ваша задача - реализовать архитектуру SPINN, а не выводить физические формулы с нуля.

SPINN - чего? 😬

Что такое SPINN

SPINN (Separable Physics-Informed Neural Network) - это архитектурная модификация PINN для многомерных задач.

Статья: [Separable Physics-Informed Neural Networks, NeurIPS 2023](#)

В обычном PINN одна большая сеть получает на вход сразу точку (x, y, z) и предсказывает значение функции $u(x, y, z)$.

В SPINN идея другая: вместо одной общей сети используются несколько небольших сетей - по одной на каждую координату.

Для нашей 3D-задачи это означает:

- первая сеть обрабатывает только x ;
- вторая сеть обрабатывает только y ;
- третья сеть обрабатывает только z .

Каждая из этих сетей возвращает вектор длины `rank`, а итоговое решение собирается через CP-подобное объединение:

$$u[i, j, k] = \sum_{r=1}^{\text{rank}} f_x[i, r] \cdot f_y[j, r] \cdot f_z[k, r].$$

Интуитивно это похоже на CP-декомпозицию ([Canonical Polyadic decomposition](#), - is%20a%20variant)), где трёхмерный тензор представляется как сумма тензоров ранга 1:

$$\hat{u} = \sum_{r=1}^R \mathbf{f}_r^{(x)} \otimes \mathbf{f}_r^{(y)} \otimes \mathbf{f}_r^{(z)}.$$

Главная идея здесь в том, что на tensor-product grid можно отдельно обработать оси, а затем собрать итоговый тензор решения. Именно поэтому SPINN особенно удобен для многомерных PDE-задач, где pointwise-подход быстро становится вычислительно дорогим.

В этой домашней работе мы используем **учебную реализацию ключевой идеи SPINN**: separable body-сети + CP-merge. Вам не нужно воспроизводить все технические детали статьи; важно корректно реализовать архитектуру и получить рабочее решение.

Но если вам захочется погрузиться в задачу максимально - препятствовать не будем, скорее замолвим за вас словечко в [LAMBDA](#).

Уравнение Гельмгольца

В задаче рассматривается 3D-уравнение Гельмгольца:

$$\nabla^2 u + k^2 u = q(x, y, z), \quad (x, y, z) \in [-1, 1]^3, \quad u|_{\text{граница}} = 0.$$

Здесь:

- $u(x, y, z)$ - неизвестная функция;
- k - волновое число;
- $q(x, y, z)$ - правая часть (источник);
- на границе области заданы нулевые граничные условия Дирихле.

Для тестирования используется точное решение:

$$u(x, y, z) = \sin(\pi x) \sin(\pi y) \sin(\pi z).$$

На всех шести гранях куба $\Omega = [-1, 1]^3$ задаются нулевые граничные условия:

$$u(x, y, z) = 0, \quad (x, y, z) \in \partial\Omega.$$

Описание задания

В этой части вам нужно:

1. реализовать три независимые body-сети — по одной для каждой координаты;
2. получить из них separable-представление решения;
3. корректно объединить выходы через CP-merge;
4. реализовать вычисление модели на решётке и на произвольных точках;
5. обучить модель, оценить качество решения на тестовой сетке и проанализировать результаты.

Несколько вспомогательных функций для точного решения, оценки ошибки и визуализации:

```
In [3]: def helmholtz3d_exact_u(a1, a2, a3, x, y, z):
        return (
            torch.sin(a1 * torch.pi * x)
            * torch.sin(a2 * torch.pi * y)
            * torch.sin(a3 * torch.pi * z)
        )

def relative_l2(u_pred, u_true):
    """Относительная L2-ошибка: ||u_pred - u_true|| / ||u_true||"""
    return torch.linalg.norm(u_pred - u_true) / torch.linalg.norm(u_true)
```

А тут вспомогательные функции для визуализации и оценки: код уже готов и нужен вам как инфраструктура для экспериментов. Он помогает визуализировать решение, отслеживать динамику loss'ов, считать ошибку на тестовой сетке и пересэмплировать коллокационные точки.

```
In [4]: def plot_helmholtz3d(x, y, z, u):
        x = x.detach().cpu().numpy().flatten()
        y = y.detach().cpu().numpy().flatten()
        z = z.detach().cpu().numpy().flatten()
        u = u.detach().cpu().numpy().flatten()

        u_norm = (u - u.min()) / (u.max() - u.min() + 1e-10)

        fig = plt.figure(figsize=(10, 8))
        ax = fig.add_subplot(111, projection="3d")

        scatter = ax.scatter(
            x, y, z,
            c=u_norm,
            s=1,
            cmap="seismic",
            vmin=0,
            vmax=1,
            alpha=0.75,
            edgecolors="none",
        )

        ax.set_title("U(x, y, z)", fontsize=18)
        ax.set_xlabel("x")
        ax.set_ylabel("y")
        ax.set_zlabel("z")

        cbar = plt.colorbar(scatter, shrink=0.8, aspect=15)
        cbar.set_label("u(x,y,z)")

        plt.tight_layout()
        plt.show()

def plot_losses(
    loss_history,
    residual_history,
```

```

boundary_history,
error_history,
log_iter,
title_prefix="",
):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    ax1.semilogy(loss_history, label="Total Loss", linewidth=2)
    ax1.semilogy(residual_history, label="PDE Loss", alpha=0.7)
    ax1.semilogy(boundary_history, label="BC Loss", alpha=0.7)
    ax1.grid(True, alpha=0.3)
    ax1.legend()
    ax1.set_xlabel("Iteration")
    ax1.set_ylabel("Loss (log scale)")
    ax1.set_title(f"{title_prefix}Training Losses")

    iterations = np.arange(0, len(error_history) * log_iter, log_iter)
    ax2.semilogy(iterations, error_history, "r-", linewidth=2, label="Relative L2 Error")
    ax2.grid(True, alpha=0.3)
    ax2.legend()
    ax2.set_xlabel("Iteration")
    ax2.set_ylabel("Error (log scale)")
    ax2.set_title(f"{title_prefix}Relative L2 Error")

    plt.tight_layout()
    plt.show()

def compute_error_on_grid(model, x_grid, y_grid, z_grid, u_true):
    """
    Вычисление ошибки модели на фиксированной сетке.
    """

    was_training = model.training
    model.eval()

    with torch.no_grad():
        coords = torch.stack(
            [x_grid.flatten(), y_grid.flatten(), z_grid.flatten()],
            dim=1,
        )
        u_pred = model(coords)
        error = relative_l2(u_pred, u_true.flatten())

    if was_training:
        model.train()

    return error, u_pred.reshape(x_grid.shape)

def resample_collocation_points(model, NC, domain_size=1.0):
    """
    Ресэмплирование коллокационных точек на кубической решётке.
    """

    N_per_axis = int(round(NC ** (1 / 3)))

    x = torch.linspace(-domain_size, domain_size, N_per_axis, device=model.device)
    y = torch.linspace(-domain_size, domain_size, N_per_axis, device=model.device)
    z = torch.linspace(-domain_size, domain_size, N_per_axis, device=model.device)

```

```
xx, yy, zz = torch.meshgrid(x, y, z, indexing="ij")

xc = xx.flatten().requires_grad_(True)
yc = yy.flatten().requires_grad_(True)
zc = zz.flatten().requires_grad_(True)

return xc, yc, zc
```

Глянем на визуализацию точного решения на тестовой сетке.

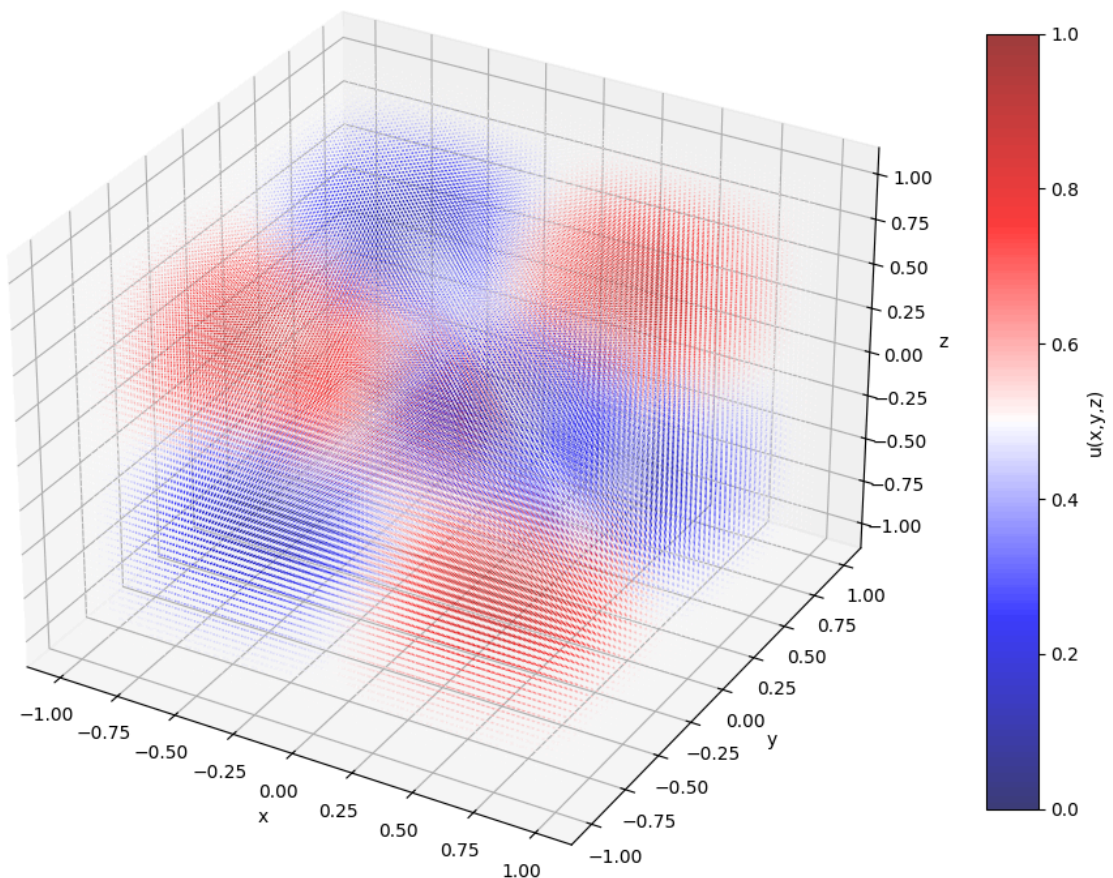
```
In [5]: N_test = 50

x_test = torch.linspace(-1, 1, N_test, device=device)
y_test = torch.linspace(-1, 1, N_test, device=device)
z_test = torch.linspace(-1, 1, N_test, device=device)

xx_test, yy_test, zz_test = torch.meshgrid(x_test, y_test, z_test, indexing="ij")
u_test = helmholtz3d_exact_u(1, 1, 1, xx_test, yy_test, zz_test)

plot_helmholtz3d(xx_test, yy_test, zz_test, u_test)
```

$U(x, y, z)$



Задание 1.1. Реализуйте SPINN-архитектуру (2 балла)

В этом пункте нужно реализовать дочерний класс от `BaseModel`, который задаёт Separable Physics-Informed Neural Network для трёхмерного уравнения Гельмгольца.

Идея остаётся той же, что и в [статье](#): вместо одной сети, работающей сразу с тройкой координат, используется три независимые body-сети - для x , y и z . Их выходы затем объединяются в separable-представление решения

$$u(x, y, z) = \sum_{j=1}^r f_j^{(1)}(x) f_j^{(2)}(y) f_j^{(3)}(z).$$

Базовый класс уже содержит всю постановку задачи: точное решение, правую часть уравнения, граничные условия, вычисление `PDE loss` и `BC loss`. Менять его не нужно. Ваша часть работы - архитектура.

Что именно требуется:

- собрать класс `BaseSPINN`;
- реализовать три независимые body-сети;
- определить, как из их выходов получается решение на полной решётке;
- отдельно продумать, как считать модель на произвольном наборе точек, когда координаты уже не образуют декартово произведение.

Здесь важно не просто получить рабочий код, а аккуратно разобраться в размерностях. От того, какие формы имеют выходы body-сетей, зависит и корректность `forward`, и вычисление `PDE loss`.

После реализации сделайте короткую sanity check-проверку:

- создайте маленькую модель;
- проверьте формы выходов body-сетей;
- убедитесь, что на решётке получается тензор формы `(Nx, Ny, Nz)`;
- отдельно проверьте вычисление на наборе произвольных точек.

In [6]: *# здесь ничего не меняем - изучаем, переиспользуем*

```
class BaseModel(nn.Module):
    """
    Базовый класс для 3D-уравнения Гельмгольца на кубе [-1, 1]^3.

    Здесь уже реализованы:
    - точное решение и правая часть PDE;
    - граничные точки на 6 гранях куба;
    - вычисление PDE loss и BC loss.

    В этом задании предполагается, что студент реализует только
    архитектурную часть в наследнике BaseSPINN.
    """

    def __init__(
        self,
        x_min: float = -1.0,
        x_max: float = 1.0,
        y_min: float = -1.0,
        y_max: float = 1.0,
        z_min: float = -1.0,
        z_max: float = 1.0,
        k: float = 1.0,
```

```

a1: int = 1,
a2: int = 1,
a3: int = 1,
N_bc: int = 32,
device: torch.device = None,
):
    super().__init__()
    self.x_min, self.x_max = x_min, x_max
    self.y_min, self.y_max = y_min, y_max
    self.z_min, self.z_max = z_min, z_max
    self.N_bc = N_bc
    self.k = k
    self.a1, self.a2, self.a3 = a1, a2, a3
    self.device = device or torch.device("cuda" if torch.cuda.is_available()

    self.mse = nn.MSELoss(reduction="mean")
    self._init_boundary_points()

def _init_boundary_points(self):
    N = self.N_bc
    lin_x = torch.linspace(self.x_min, self.x_max, N)
    lin_y = torch.linspace(self.y_min, self.y_max, N)
    lin_z = torch.linspace(self.z_min, self.z_max, N)

    faces = []

    for x_val in [self.x_min, self.x_max]:
        yy, zz = torch.meshgrid(lin_y, lin_z, indexing="ij")
        xx = torch.full_like(yy, x_val)
        faces.append(torch.stack([xx.reshape(-1), yy.reshape(-1), zz.reshape(-1)]))

    for y_val in [self.y_min, self.y_max]:
        xx, zz = torch.meshgrid(lin_x, lin_z, indexing="ij")
        yy = torch.full_like(xx, y_val)
        faces.append(torch.stack([xx.reshape(-1), yy.reshape(-1), zz.reshape(-1)]))

    for z_val in [self.z_min, self.z_max]:
        xx, yy = torch.meshgrid(lin_x, lin_y, indexing="ij")
        zz = torch.full_like(xx, z_val)
        faces.append(torch.stack([xx.reshape(-1), yy.reshape(-1), zz.reshape(-1)]))

    for i, coords in enumerate(faces, 1):
        self.register_buffer(f"coords_bc{i}", coords.to(self.device))
        self.register_buffer(
            f"zeros_bc{i}",
            torch.zeros(coords.shape[0], 1, device=self.device)
        )

def reference_solution(self, x, y, z):
    """Точное решение. Используется только для оценки ошибки."""
    return (
        torch.sin(self.a1 * np.pi * x)
        * torch.sin(self.a2 * np.pi * y)
        * torch.sin(self.a3 * np.pi * z)
    )

def function_of_source(self, x, y, z):
    """Правая часть PDE, полученная подстановкой точного решения."""
    u = self.reference_solution(x, y, z)
    lap_coef = -(self.a1 * np.pi) ** 2 - (self.a2 * np.pi) ** 2 - (self.a3 * np.pi) ** 2
    return u - lap_coef * u

```

```

        return (lap_coef + self.k**2) * u

def get_body_outputs(self, x, y, z):
    """
    Должен быть реализован в наследнике.

    Ожидаемые формы:
    - x: (Nx,), y: (Ny,), z: (Nz,)
    - f_x: (Nx, rank), f_y: (Ny, rank), f_z: (Nz, rank)
    """
    raise NotImplementedError

def forward(self, x, y, z):
    """
    Значения решения на полной декартовой решётке.
    """
    raise NotImplementedError

def forward_pointwise(self, coords):
    """
    Значения решения на произвольном наборе точек coords формы (B, 3).
    """
    raise NotImplementedError

@staticmethod
def _body_second_deriv(f, inp):
    """
    Вторая производная по одной координате для каждого канала ранга.
    Предполагается, что f.shape == (N, rank), inp.shape == (N,).
    """
    _, rank = f.shape
    d2 = torch.zeros_like(f)

    for r in range(rank):
        d1 = torch.autograd.grad(f[:, r].sum(), inp, create_graph=True)[0]
        d2[:, r] = torch.autograd.grad(d1.sum(), inp, create_graph=True)[0]

    return d2

def calculate_pde_loss(self, x_col, y_col, z_col):
    """
    PDE loss на коллокационной решётке.
    """
    x = x_col.clone().detach().requires_grad_(True)
    y = y_col.clone().detach().requires_grad_(True)
    z = z_col.clone().detach().requires_grad_(True)

    f_x, f_y, f_z = self.get_body_outputs(x, y, z)

    f_x_d2 = self._body_second_deriv(f_x, x)
    f_y_d2 = self._body_second_deriv(f_y, y)
    f_z_d2 = self._body_second_deriv(f_z, z)

    u = self.forward(x, y, z)

    lap = (
        torch.einsum("ir,jr,kr->ijk", f_x_d2, f_y, f_z)
        + torch.einsum("ir,jr,kr->ijk", f_x, f_y_d2, f_z)
        + torch.einsum("ir,jr,kr->ijk", f_x, f_y, f_z_d2)
    )

```

```

xx, yy, zz = torch.meshgrid(x, y, z, indexing="ij")
q = self.function_of_source(xx, yy, zz)

residual = lap + self.k**2 * u - q
return self.mse(residual, torch.zeros_like(residual))

def calculate_bc_loss(self):
    """
    MSE на шести гранях куба.
    """
    loss = torch.tensor(0.0, device=self.device)

    for i in range(1, 7):
        coords = getattr(self, f"coords_bc{i}")
        zeros = getattr(self, f"zeros_bc{i}")
        u_bc = self.forward_pointwise(coords).unsqueeze(-1)
        loss = loss + self.mse(u_bc, zeros)

    return loss

def calculate_loss(self, x_col, y_col, z_col):
    loss_pde = self.calculate_pde_loss(x_col, y_col, z_col)
    loss_bc = self.calculate_bc_loss()
    return loss_pde, loss_bc

```

Теперь реализуйте класс `BaseSPINN`.

В этом пункте сигнатуры методов не даны заранее. Вам нужно самостоятельно восстановить архитектуру класса, согласовав её с тем, как устроен `BaseModel`, как вычисляется `PDE loss`, и как далее используется модель в обучении.

Вам нужно самостоятельно продумать:

- как устроена одна body-сеть;
- какие именно модули удобно использовать для её сборки;
- как объединять выходы трёх ветвей на полной решётке;
- чем вычисление на решётке отличается от вычисления в отдельных точках.

Перед обучением имеет смысл убедиться, что ваша реализация согласована с остальным кодом ноутбука и возвращает тензоры ожидаемых размерностей.

```

In [7]: class BaseSPINN(BaseModel):
        """
        Separable PINN для 3D-задачи.

        Нужно реализовать:
        - __init__
        - _make_body_net
        - get_body_outputs
        - forward
        - forward_pointwise
        """

        def __init__(
            self,
            rank = 16,

```

```

        hidden_dim = 64,
        n_hidden_layers = 4,
        N_bc = 32,
        k = 1.0,
        a1 = 1,
        a2 = 1,
        a3 = 1,
        device: torch.device = None,
    ):
        super().__init__(N_bc=N_bc, k=k, a1=a1, a2=a2, a3=a3, device=device)
        self.rank = rank
        self.hidden_dim = hidden_dim
        self.n_hidden_layers = n_hidden_layers

        self.body_x = self._make_body_net()
        self.body_y = self._make_body_net()
        self.body_z = self._make_body_net()

    def _make_body_net(self):
        layers = []
        layers.append(nn.Linear(1, self.hidden_dim))
        layers.append(nn.Tanh())
        for _ in range(self.n_hidden_layers - 1):
            layers.append(nn.Linear(self.hidden_dim, self.hidden_dim))
            layers.append(nn.Tanh())
        layers.append(nn.Linear(self.hidden_dim, self.rank))
        return nn.Sequential(*layers)

    def get_body_outputs(self, x, y, z):
        f_x = self.body_x(x.unsqueeze(-1))
        f_y = self.body_y(y.unsqueeze(-1))
        f_z = self.body_z(z.unsqueeze(-1))
        return f_x, f_y, f_z

    def forward(self, x, y, z):
        f_x, f_y, f_z = self.get_body_outputs(x, y, z)
        u = torch.einsum("ir,jr,kr->ijk", f_x, f_y, f_z)
        return u

    def forward_pointwise(self, coords: torch.Tensor):
        x = coords[:, 0]
        y = coords[:, 1]
        z = coords[:, 2]
        f_x, f_y, f_z = self.get_body_outputs(x, y, z)
        u = torch.sum(f_x * f_y * f_z, dim=1, keepdim=True)
        return u

def init_tanh_(m: nn.Module):
    """
    Реализуйте инициализацию весов для линейных слоёв.
    """
    if isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight)
        if m.bias is not None:
            nn.init.zeros_(m.bias)

```

Ниже приведён готовый код для обучения и оценки. Его можно использовать как есть.

В экспериментах имеет смысл варьировать архитектурные параметры модели и параметры вызова `train_spinn`, но переписывать сам цикл обучения здесь не требуется.

```
In [8]: def make_collocation_grid(N_per_axis, domain_size=1.0, device="cuda"):
        """
        Возвращает оси для декартовой коллокационной решётки
        Полная 3D-решётка содержит  $N_{\text{per\_axis}}^3$  точек
        """
        x = torch.linspace(-domain_size, domain_size, N_per_axis, device=device, req
        y = torch.linspace(-domain_size, domain_size, N_per_axis, device=device, req
        z = torch.linspace(-domain_size, domain_size, N_per_axis, device=device, req
        return x, y, z

    def compute_error_on_grid(model, x_grid, y_grid, z_grid, u_true):
        was_training = model.training
        model.eval()

        with torch.no_grad():
            u_pred = model(x_grid[:, 0, 0], y_grid[0, :, 0], z_grid[0, 0, :])
            error = relative_l2(u_pred, u_true)

        if was_training:
            model.train()

        return error, u_pred

    def train_spinn(
        model,
        optimizer,
        x_test,
        y_test,
        z_test,
        u_test,
        epochs=10000,
        log_iter=100,
        resample_iter=100,
        N_per_axis=64,
        domain_size=1.0,
        device="cuda",
        scheduler=None,
        pde_weight=1.0,
        bc_weight=1.0,
    ):
        history = {
            "loss": [],
            "residual": [],
            "boundary": [],
            "error": [],
        }

        best_error = float("inf")
        best_u = None

        is_lbfgs = isinstance(optimizer, torch.optim.LBFGS)

        pbar = trange(1, epochs + 1)

        for epoch in pbar:
```

```

if epoch == 1 or epoch % resample_iter == 0:
    xc, yc, zc = make_collocation_grid(N_per_axis, domain_size, device)

if is_lbfgs:
    def closure():
        optimizer.zero_grad()
        loss_pde, loss_bc = model.calculate_loss(xc, yc, zc)
        loss = pde_weight * loss_pde + bc_weight * loss_bc
        loss.backward()
        return loss
    loss = optimizer.step(closure)
    loss_pde, loss_bc = model.calculate_loss(xc, yc, zc)
    loss_val = (pde_weight * loss_pde + bc_weight * loss_bc).item()

else:
    optimizer.zero_grad()

    loss_pde, loss_bc = model.calculate_loss(xc, yc, zc)
    loss = pde_weight * loss_pde + bc_weight * loss_bc

    loss.backward()
    optimizer.step()

if scheduler is not None and not is_lbfgs:
    scheduler.step(loss_val)

history["loss"].append(loss.item())
history["residual"].append(loss_pde.item())
history["boundary"].append(loss_bc.item())

if epoch % log_iter == 0:
    error, u_pred = compute_error_on_grid(model, x_test, y_test, z_test,
    history["error"].append(error.item())

    if error < best_error:
        best_error = error
        best_u = u_pred.detach().clone()

    display.clear_output(wait=True)
    plot_losses(
        history["loss"],
        history["residual"],
        history["boundary"],
        history["error"],
        log_iter,
    )

    if error <= best_error:
        print(f"New best: rel L2 = {best_error:.2e}")
        plot_helmholtz3d(x_test, y_test, z_test, u_pred)

    pbar.set_description(
        f"L={loss.item():.2e} | PDE={loss_pde.item():.2e} | "
        f"BC={loss_bc.item():.2e} | err={error.item():.2e} | best={best_
    )

print(f"\nDone. Best relative L2 error: {best_error:.2e}")

display.clear_output(wait=True)
plot_losses(

```

```

        history["loss"],
        history["residual"],
        history["boundary"],
        history["error"],
        log_iter,
        title_prefix="Final ",
    )

    if best_u is not None:
        plot_helmholtz3d(x_test, y_test, z_test, best_u)

    return history, best_error, best_u

```

Перед обучением полезно убедиться, что реализация ведёт себя так, как вы ожидаете. Для этого достаточно короткой проверки размерностей на небольшом примере.

In [9]: *# Sanity check*

```

model_test = BaseSPINN(
    rank=4,
    hidden_dim=16,
    n_hidden_layers=2,
    N_bc=8,
    k=1.0,
    a1=1, a2=1, a3=1,
    device=device
).to(device)

Nx, Ny, Nz = 10, 12, 14
x_axis = torch.linspace(-1, 1, Nx, device=device)
y_axis = torch.linspace(-1, 1, Ny, device=device)
z_axis = torch.linspace(-1, 1, Nz, device=device)

fx, fy, fz = model_test.get_body_outputs(x_axis, y_axis, z_axis)
print(f"fx: {fx.shape}, fy: {fy.shape}, fz: {fz.shape}")

u_grid = model_test.forward(x_axis, y_axis, z_axis)
print(f"u_grid shape: {u_grid.shape}")

coords = torch.rand(20, 3, device=device) * 2 - 1
u_points = model_test.forward_pointwise(coords)
print(f"u_points shape: {u_points.shape}")

xc, yc, zc = resample_collocation_points(model_test, NC=512, domain_size=1.0)
loss_pde, loss_bc = model_test.calculate_loss(xc, yc, zc)
print(f"PDE loss: {loss_pde.item():.4e}, BC loss: {loss_bc.item():.4e}")
optimizer = torch.optim.Adam(model_test.parameters(), lr=1e-3)
loss = loss_pde + loss_bc
loss.backward()
optimizer.step()

N_test = 50
x_test = torch.linspace(-1, 1, N_test, device=device)
y_test = torch.linspace(-1, 1, N_test, device=device)
z_test = torch.linspace(-1, 1, N_test, device=device)
xx_test, yy_test, zz_test = torch.meshgrid(x_test, y_test, z_test, indexing="ij")
u_true = helmholtz3d_exact_u(1, 1, 1, xx_test, yy_test, zz_test)

```



```

error, u_pred = compute_error_on_grid(model_test, xx_test, yy_test, zz_test, u_t)
print(f"Текущая относительная L2 ошибка (до обучения): {error.item():.4f}")

plot_helmholtz3d(xx_test, yy_test, zz_test, u_pred)

```

```

fx: torch.Size([10, 4]), fy: torch.Size([12, 4]), fz: torch.Size([14, 4])
u_grid shape: torch.Size([10, 12, 14])
u_points shape: torch.Size([20, 1])

```

```

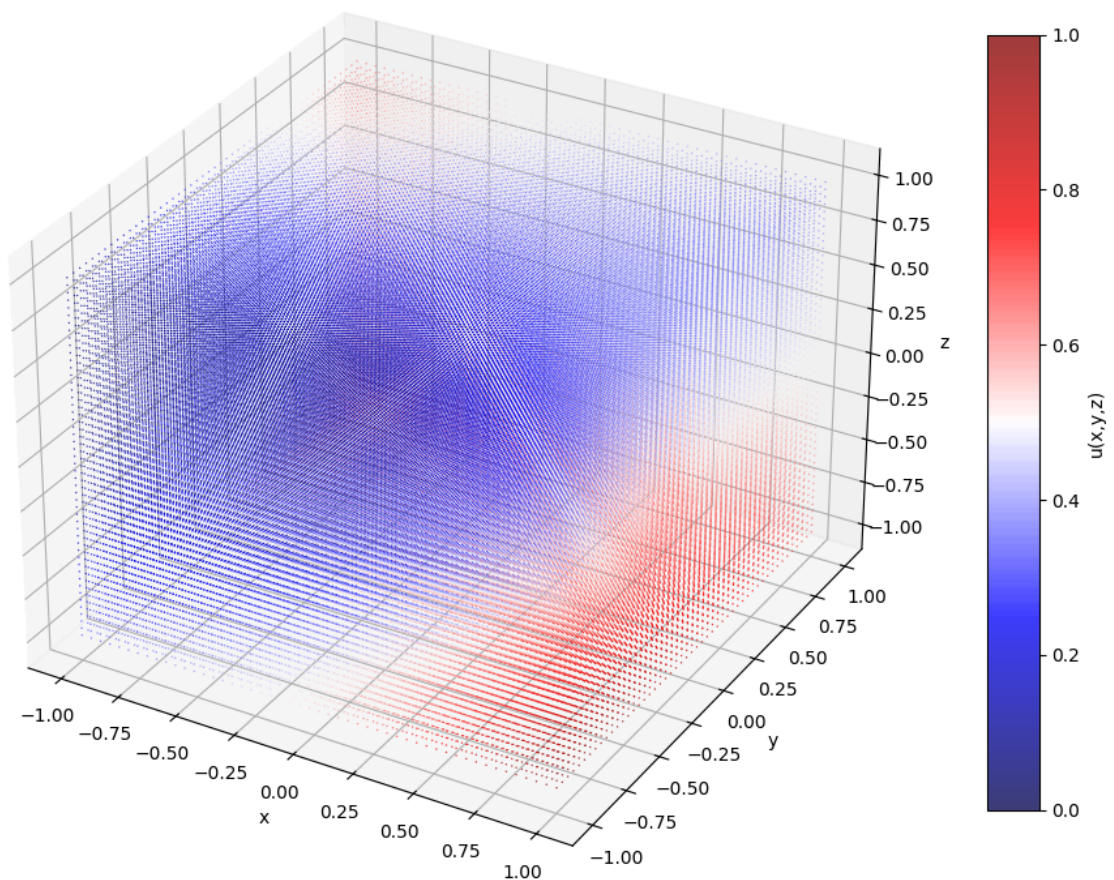
c:\Python Projects\Jupyter\.venv\Lib\site-packages\torch\nn\modules\loss.py:626:
UserWarning: Using a target size (torch.Size([64, 1])) that is different to the i
nput size (torch.Size([64, 1, 1])). This will likely lead to incorrect results du
e to broadcasting. Please ensure they have the same size.
    return F.mse_loss(input, target, reduction=self.reduction)

```

PDE loss: 6.8534e+01, BC loss: 9.0292e-03

Текущая относительная L2 ошибка (до обучения): 1.0039

$U(x, y, z)$



Задание 1.2. Обучите модель (1,5 балла)

Теперь пора подобрать рабочую конфигурацию и обучить модель.

В этой части важно получить устойчиво обучающуюся модель и добиться разумного качества на тестовой сетке.

На сходимость обычно влияют:

- ранг `rank` ;

- ширина и глубина body-сетей;
- плотность коллокационной решётки;
- выбор оптимизатора и шага обучения;
- баланс между `PDE loss` и `BC loss`.

Отдельно полезно следить за тем, как меняется поведение модели при увеличении `rank`: рост качества здесь часто сопровождается заметным ростом времени обучения.

In [20]: `# YOUR CODE: выберите конфигурацию модели, оптимизатор и параметры обучения`
`# `(\! \!)/``

```
EPOCHS = 100
LOG_ITER = 10
RESAMPLE_ITER = 20
N_PER_AXIS = 48

model = BaseSPINN(
    rank=24,
    hidden_dim=48,
    n_hidden_layers=4,
    N_bc=48,
    k=1.0,
    a1=1,
    a2=1,
    a3=1,
    device=device,
).to(device)

model.apply(init_tanh_)

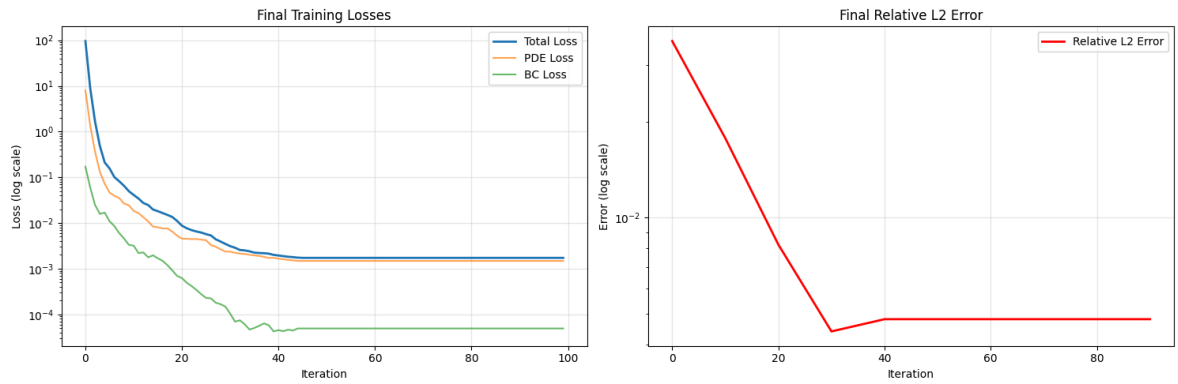
optimizer = torch.optim.LBFGS(
    model.parameters(),
    lr=1.0,
    max_iter=50,
    history_size=100,
    line_search_fn="strong_wolfe",
    tolerance_grad=1e-16,
    tolerance_change=1e-16
)
```

In [21]: `history, best_error, best_u = train_spinn(`

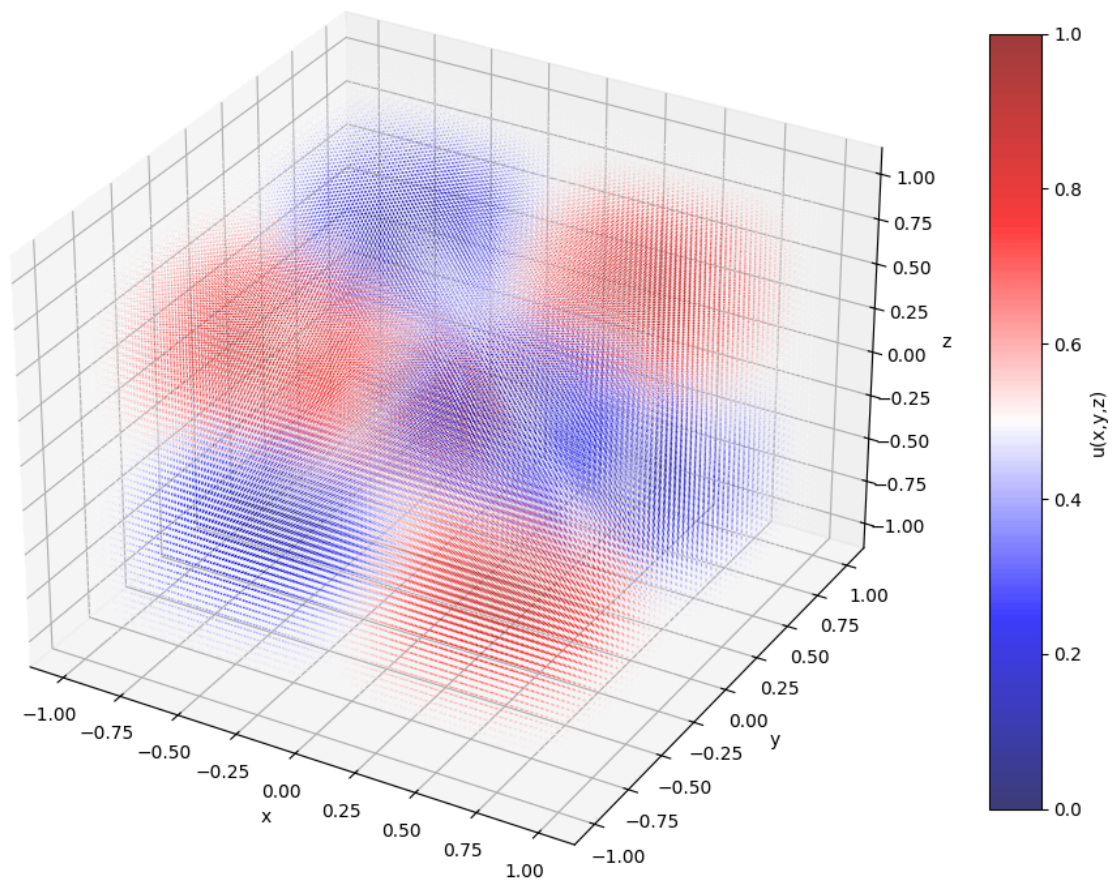
```
    model=model,
    optimizer=optimizer,
    x_test=xx_test,
    y_test=yy_test,
    z_test=zz_test,
    u_test=u_test,
    epochs=EPOCHS,
    log_iter=LOG_ITER,
    resample_iter=RESAMPLE_ITER,
    N_per_axis=N_PER_AXIS,
    domain_size=1.0,
    device=device,
    scheduler=None,
    pde_weight=1.0,
```

bc_weight=5.0,

)



$U(x, y, z)$



После обучения посчитайте итоговые метрики на тестовой сетке.

Подбирайте качество за счёт корректной архитектуры, выбора гиперпараметров и настройки обучения, не используя прямую подгонку под тестовую сетку. За такое бан.

```
In [22]: model.eval()
with torch.no_grad():
    u_pred_final = model(xx_test[:, 0, 0], yy_test[0, :, 0], zz_test[0, 0, :])

mse = ((u_test.cpu().numpy() - u_pred_final.cpu().numpy()) ** 2).mean()
rel_l2 = relative_l2(u_pred_final, u_test).item()
```

```
print(f"MSE: {mse:.2e}")
print(f"Relative L2: {rel_l2:.2e}")
```

MSE: 2.72e-06
Relative L2: 4.81e-03

```
In [23]: score_1_2 = 0.0
if mse <= 5e-3:
    score_1_2 = 0.5 * 1.5
if mse <= 5e-4:
    score_1_2 = 0.6 * 1.5
if mse <= 1e-5:
    score_1_2 = 0.7 * 1.5
if mse <= 1e-7:
    score_1_2 = 0.9 * 1.5
if mse <= 5e-8:
    score_1_2 = 1.0 * 1.5

print(f"Баллы за пункт 1.2: {score_1_2:.2f} / 1.5")
```

Баллы за пункт 1.2: 1.05 / 1.5

Оценка за пункт 1.2 зависит от наилучшего достигнутого порога по `mse`: каждому порогу соответствует своя доля из **1,5 балла** за качество. Итоговый балл определяется по самому строгому из выполненных условий.

Примечание. Итоговые пороги качества для этой части будут проверяться по метрикам на тестовой сетке. Не забывайте сохранить веса лучшей модели: они понадобятся при сдаче домашнего задания.

```
In [91]: torch.save(model.state_dict(), "best_spinn_model.pth")
```

Задание 1.3. Краткий анализ (0,5 балла)

После обучения кратко опишите, как вы подбирали конфигурацию модели и что наблюдали в экспериментах.

Можно обсудить:

1. С каких параметров вы начали и к какой конфигурации пришли в итоге.
2. Как изменение `rank` влияло на качество и время обучения.
3. Что сильнее влияло на результат: архитектура body-сетей, плотность коллокационной решётки или баланс между `PDE loss` и `BC loss`.
4. Были ли признаки того, что модель реализована некорректно или обучается неустойчиво, и как вы это диагностировали.

1. Начал с optimizer Adam rank = 32 hidden_dim = 64 n_hidden_layers = 4 N_bc = 64 EPOCHS = 1000 LOG_ITER = 200 RESAMPLE_ITER = 500 N_PER_AXIS = 64 pde_weight = 1.0 bc_weight = 1.0 Закончил с тем, что сейчас в коде выше.
2. Чем больше rank, тем выше качество получалось. Также сильно возрастало время обучения, что очень мешало делать этот номер.

3. `hidden_dim` не особо помогала, возможно потому что я сразу задал большое значение. `N_per_axis` довольно сильно влияло на скорость выполнения итераций, но не давало особо импакта. `N_bc` улучшило точность и не особо сильно замедлило обучение. `bc_weight` не особо экспериментировал, но на 5 мне показались результаты более точными, чем на 1, а в скорости особо не проиграл.
4. Были моменты, когда модель обучалась неустойчиво. Значение MSE и L2 могли очень сильно отличаться при одних и тех же параметрах, я ставил протсто больше эпох. Еще я перешел на новый оптимайзер, поскольку оказался в тупике с Adam, а этот мне показался более эффективным и давал скоры совсем близкие к трешхолду на 1.35, но не позволил его достичь.

Часть 2. Классификация звуков UrbanSound8K с помощью CNN (6 баллов)

Разбалловка

Во второй части нужно пройти весь стандартный пайплайн аудиоклассификации: подготовить данные, реализовать модель, обучить её и провести итоговую оценку.

Подзадача	Баллы	Что оценивается
2.1 Подготовка данных и Dataset	1,0	Корректный fold-aware pipeline, реализация <code>UrbanSoundDataset</code> , аккуратная работа с метками
2.2 Архитектура CNN	1,0	Рабочая сверточная модель для классификации мел-спектрограмм
2.3 Обучение и валидация	1,0	Реализация train/validate, early stopping, осмысленная настройка обучения
2.4 Качество модели	2,5	Итоговое качество по accuracy
2.5 Краткий анализ	0,5	Наблюдения по архитектуре, аугментациям, ошибкам модели и качеству

В этой части важно не только получить хороший score, но и корректно организовать эксперимент. Использование правильного протокола разбиения данных здесь является частью задания, а не технической деталью.

Введение в обработку аудио

Почему не всегда удобно работать с сырым аудиосигналом?

Аудиосигнал - это последовательность амплитудных отсчётов во времени. Например, 4-секундный фрагмент при частоте дискретизации 22 050 Гц содержит около 88 тысяч значений. Теоретически такие данные можно подавать напрямую в нейросеть, но на практике это не самый удобный вариант:

1. **Высокая размерность.** Даже короткий сигнал содержит много отсчётов.
2. **Сложная временная структура.** В сигнале одновременно важны и короткие локальные паттерны, и более длинные зависимости.
3. **Частотная информация скрыта неявно.** По самому временному ряду не так удобно понять, какие частоты присутствуют в сигнале.

Спектрограммы

Это представление сигнала в плоскости "время–частота". Она строится с помощью кратковременного преобразования Фурье (STFT): сигнал разбивается на короткие окна, для каждого окна вычисляется частотный спектр, после чего эти спектры собираются по времени.

В результате получается 2D-представление, где:

- по оси X идёт время;
- по оси Y - частота;
- цвет отражает энергию сигнала на данной частоте в данный момент.

Мел-спектрограмма

Обычная спектрограмма полезна, но не полностью соответствует тому, как человек воспринимает звук. Поэтому в задачах аудиоклассификации часто используют мел-спектрограммы.

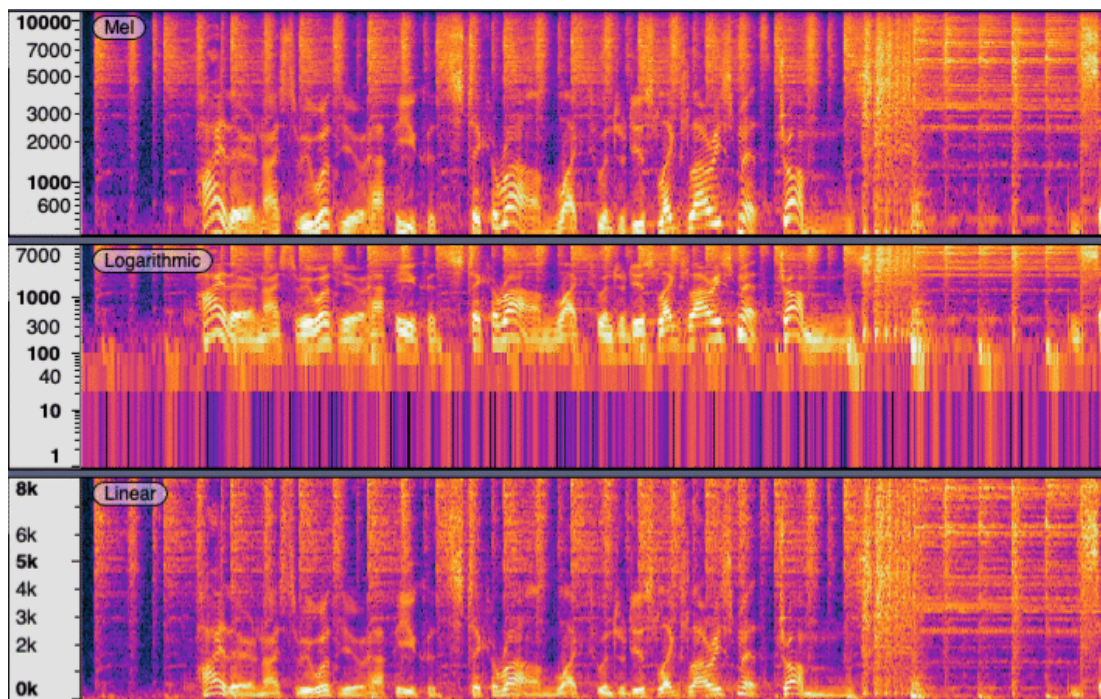
Идея здесь в том, что:

- частоты переводятся в мел-шкалу, которая лучше соответствует человеческому слуху;
- амплитуды обычно переводятся в децибельную шкалу, то есть в логарифмический масштаб.

Мел-спектрограммы удобны по нескольким причинам:

1. уменьшают размерность;
2. сохраняют наиболее важную перцептивную информацию;
3. дают 2D-представление, с которым удобно работать при помощи CNN.

Именно поэтому в этой части домашнего задания мы будем классифицировать не сырые аудиосигналы, а именно - мел-спектрограммы.



Пример мел-спектрограммы: ось X - время, ось Y - частота по мел-шкале, цвет отражает амплитуду (от тёмных = низкая энергия до ярких = высокая энергия)

Источники - [курс DLA](#), [audacity](#)

UrbanSound8K

Мы будем работать с датасетом **UrbanSound8K**. В нём содержится 8732 размеченных аудиофрагмента городских звуков длительностью не более 4 секунд. В датасете 10 классов: `air_conditioner`, `car_horn`, `children_playing`, `dog_bark`, `drilling`, `engine_idling`, `gun_shot`, `jackhammer`, `siren`, `street_music`.

Скачиваем UrbanSound8K отсюда

<https://urbansounddataset.weebly.com/urbansound8k.html>

Структура датасета:

```
UrbanSound8K/
├── audio/
│   ├── fold1/
│   ├── fold2/
│   └── ...
└── metadata/
    └── UrbanSound8K.csv
```

Важное замечание про протокол разбиения

UrbanSound8K уже разбит авторами на 10 fold'ов, и в этой задаче нужно использовать именно их.

Для этого датасета не стоит делать случайное разбиение, потому что фрагменты, относящиеся к одному и тому же исходному событию, могут оказаться и в обучении, и в тесте. В таком случае качество будет завышено и перестанет честно отражать способность модели обобщать на новые данные.

Поэтому здесь используется следующая схема:

- для быстрых экспериментов: `fold 1-8 -> train`, `9 -> val`, `10 -> test`;
- для финальной оценки: один прогон 10-fold cross-validation по official folds.

Случайно перераспределять примеры между fold'ами не нужно.

При этом `shuffle=True` внутри `train_loader` использовать можно.

```
In [5]: DATASET_PATH = "sound_datasets/UrbanSound8K.csv"

# Загрузим метаданные
metadata = pd.read_csv(DATASET_PATH)

print(f"Всего примеров: {len(metadata)}")
print("\nРаспределение по классам:")
print(metadata["class"].value_counts().sort_index())
print("\nРаспределение по fold'ам:")
print(metadata["fold"].value_counts().sort_index())
```

Всего примеров: 8732

Распределение по классам:

```
class
air_conditioner      1000
car_horn              429
children_playing     1000
dog_bark             1000
drilling             1000
engine_idling        1000
gun_shot             374
jackhammer           1000
siren                929
street_music         1000
Name: count, dtype: int64
```

Распределение по fold'ам:

```
fold
1      873
2      888
3      925
4      990
5      936
6      823
7      838
8      806
9      816
10     837
Name: count, dtype: int64
```

Сначала посмотрим, как выглядят один и тот же аудиофрагмент в виде волновой формы, обычной спектрограммы и мел-спектрограммы.


```

In [6]: def plot_waveform_and_spectrogram(audio_path, class_name):
        y, sr = librosa.load(audio_path, sr=22050)

        fig, axes = plt.subplots(3, 1, figsize=(14, 10))

        librosa.display.waveshow(y, sr=sr, ax=axes[0])
        axes[0].set_title(f"Волновая форма – {class_name}", fontsize=14, fontweight=
        axes[0].set_xlabel("Время (с)")
        axes[0].set_ylabel("Амплитуда")
        axes[0].grid(alpha=0.3)

        D = librosa.stft(y)
        S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)
        img1 = librosa.display.specshow(
            S_db, sr=sr, x_axis="time", y_axis="hz", ax=axes[1], cmap="viridis"
        )
        axes[1].set_title("Спектрограмма", fontsize=14, fontweight="bold")
        fig.colorbar(img1, ax=axes[1], format="%+2.0f dB")

        mel_spec = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128, fmax=8000)
        mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)
        img2 = librosa.display.specshow(
            mel_spec_db, sr=sr, x_axis="time", y_axis="mel", ax=axes[2], cmap="magma"
        )
        axes[2].set_title("Мел-спектрограмма", fontsize=14, fontweight="bold")
        fig.colorbar(img2, ax=axes[2], format="%+2.0f dB")

        plt.tight_layout()
        plt.show()
        return y, sr

        random_idx = np.random.randint(0, len(metadata))
        sample = metadata.iloc[random_idx]

        audio_path = os.path.join(os.path.dirname(DATASET_PATH), f"fold{sample.fold}", s
        print(f"Sample: {sample.slice_file_name}")
        print(f"Class: {sample['class']}")
        print(f"Fold: {sample.fold}\n")

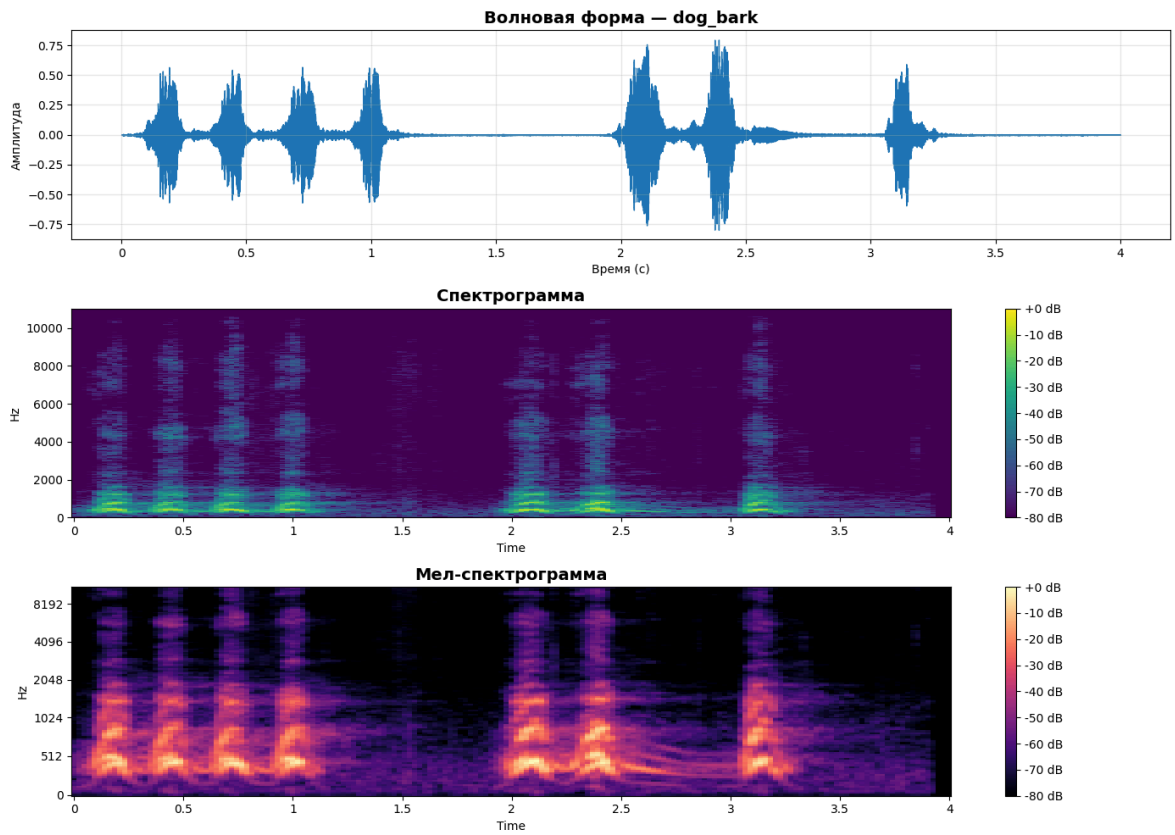
        y, sr = plot_waveform_and_spectrogram(audio_path, sample["class"])
        Audio(y, rate=sr)

```

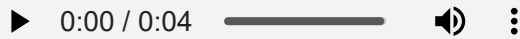
Sample: 74723-3-0-0.wav

Class: dog_bark

Fold: 4



Out[6]:



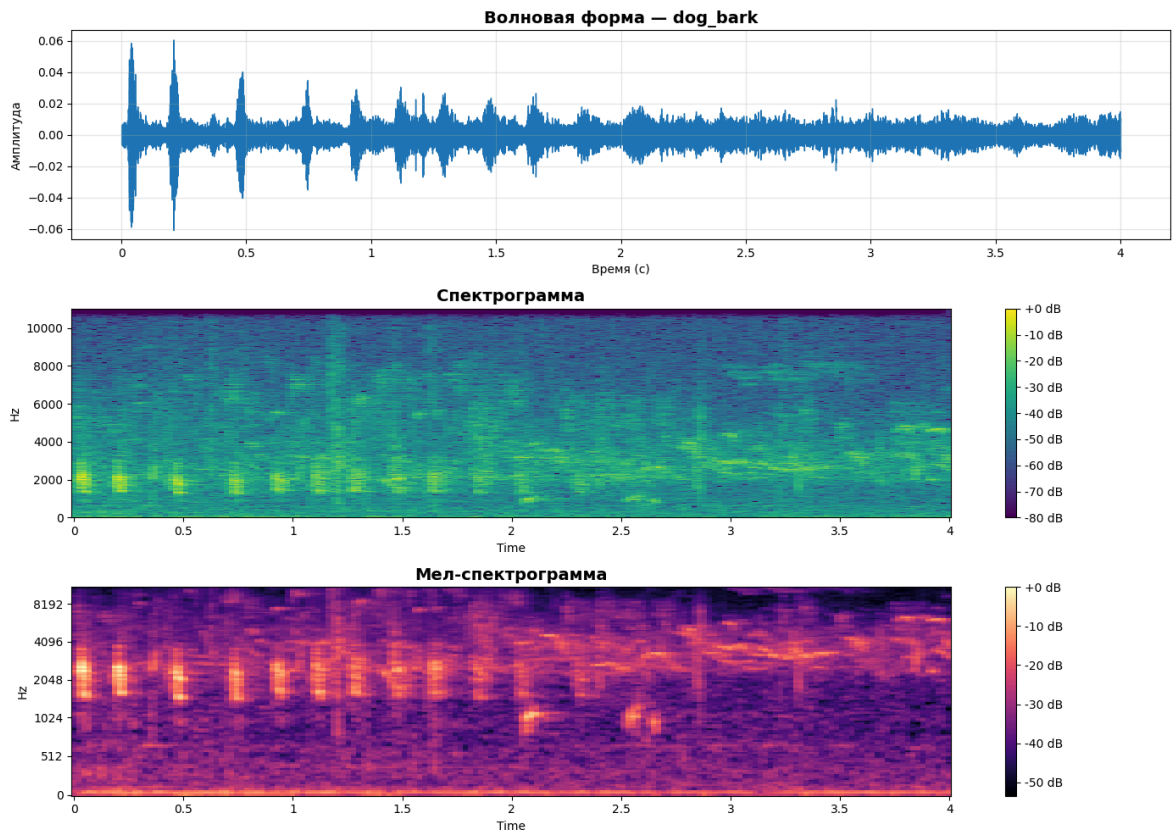
In [7]:

```
target_class = 'dog_bark'

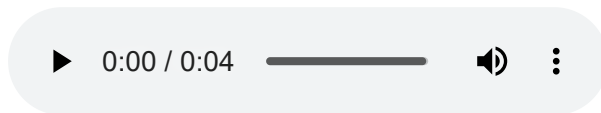
class_samples = metadata[metadata['class'] == target_class]
sample = class_samples.sample(1).iloc[0]

audio_path = os.path.join(os.path.dirname(DATASET_PATH), f"fold{sample.fold}", s
y, sr = plot_waveform_and_spectrogram(audio_path, target_class)

Audio(y, rate=sr)
```



Out[7]:



Задание 2.1. Подготовьте датасет к обучению модели (1 балл)

В этой части нужно реализовать `UrbanSoundDataset`.

Несколько важных условий:

- используйте официальные fold'ы;
- не делайте `fit` для `LabelEncoder` отдельно на каждом split;
- не извлекайте мел-спектрограммы заново при каждом обращении к `__getitem__`, если этого можно избежать.

Последний пункт особенно важен. Извлечение спектрограмм через `librosa` - CPU-bound операция, и если делать её на лету для каждого батча, обучение быстро начинает тормозить не из-за модели, а из-за препроцессинга. Поэтому в этой задаче рекомендуется один раз подготовить спектрограммы и либо держать их в памяти, либо кэшировать на диск.

Идея простая: `__getitem__` должен быть как можно легче. В идеале там уже не должно происходить ничего дорогого, кроме извлечения готового тензора и, при необходимости, несложной аугментации.

```
In [30]: # Единый LabelEncoder

ALL_CLASSES = sorted(metadata["class"].unique())
```

```

global_label_encoder = LabelEncoder()
global_label_encoder.fit(ALL_CLASSES)

print(f"Классы ({len(global_label_encoder.classes_)}):")
print(list(global_label_encoder.classes_))

```

Классы (10):

```

[np.str_('air_conditioner'), np.str_('car_horn'), np.str_('children_playing'), np.str_('dog_bark'), np.str_('drilling'), np.str_('engine_idling'), np.str_('gun_shot'), np.str_('jackhammer'), np.str_('siren'), np.str_('street_music')]

```

```

In [31]: class UrbanSoundDataset(Dataset):
    def __init__(
        self,
        metadata,
        dataset_path,
        label_encoder,
        target_length=4,
        sr=22050,
        n_mels=128,
        augment=False,
        cache_dir=None,
    ):
        """
        Реализуйте датасет для UrbanSound8K.

        Что стоит продумать:
        - как хранить metadata и переиндексировать его
        - как загружать аудио и приводить его к фиксированной длине
        - как строить и нормализовать мел-спектрограммы
        - как организовать кэш, чтобы не пересчитывать спектрограммы каждый раз
        - какие аугментации допустимо применять только на train
        """

        # your code here
        # \(\♡_♡)/

        self.metadata = metadata.reset_index(drop=True)
        self.dataset_path = dataset_path
        self.label_encoder = label_encoder
        self.target_length = target_length
        self.sr = sr
        self.n_mels = n_mels
        self.augment = augment
        self.cache_dir = cache_dir
        if cache_dir is not None:
            os.makedirs(cache_dir, exist_ok=True)
        self.target_samples = int(target_length * sr)

    def __len__(self):
        # (/🌀🌀)/*:.*°💎

        return len(self.metadata)

    def load_audio(self, file_path):
        """
        Загрузить аудио, привести к нужной частоте дискретизации
        и длине target_length
        """

```

```

# your code here ୨(*ଓ*)୧

y, orig_sr = librosa.load(file_path, sr=None)
if orig_sr != self.sr:
    y = librosa.resample(y, orig_sr=orig_sr, target_sr=self.sr)
if len(y) > self.target_samples:
    y = y[:self.target_samples]
else:
    pad_len = self.target_samples - len(y)
    y = np.pad(y, (0, pad_len), 'constant')
return y

def extract_mel_spectrogram(self, y):
    """
    Построить мел-спектрограмму и перевести её в dB-шкалу
    """

    # your code here (づ。●_●。)づ

    mel_spec = librosa.feature.melspectrogram(
        y=y, sr=self.sr, n_mels=self.n_mels,
        n_fft=1024, hop_length=512, fmax=8000
    )
    return librosa.power_to_db(mel_spec, ref=np.max)

def augment_audio(self, y):
    """
    Реализовать аугментации аудио
    Использовать только если self.augment=True
    """

    # your code here ୩(୩!୩)/
    if not self.augment:
        return y
    if np.random.random() < 0.5:
        t = y.shape[1]
        mask_len = np.random.randint(5, min(20, t))
        mask_start = np.random.randint(0, t - mask_len + 1)
        y[:, mask_start:mask_start+mask_len] = y.min()
    if np.random.random() < 0.5:
        f = y.shape[0]
        mask_len = np.random.randint(3, min(10, f))
        mask_start = np.random.randint(0, f - mask_len + 1)
        y[mask_start:mask_start+mask_len, :] = y.min()
    return y

def __getitem__(self, idx):
    """
    Вернуть пару (mel_spectrogram, label)
    """

    # your code here ୩^•୧•^୩

    row = self.metadata.iloc[idx]
    fold = row['fold']
    filename = row['slice_file_name']
    file_path = os.path.join(self.dataset_path, f'fold{fold}', filename)

    cache_name = f"{filename}.npy"

```

```

cache_path = os.path.join(self.cache_dir, cache_name) if self.cache_dir

if cache_path and os.path.exists(cache_path):
    mel = np.load(cache_path)
else:
    y = self.load_audio(file_path)
    mel = self.extract_mel_spectrogram(y)
    if cache_path:
        np.save(cache_path, mel)

mel = self.augment_audio(mel)

label = row['classID']
mel_tensor = torch.from_numpy(mel).float().unsqueeze(0)
return mel_tensor, label

```

```

In [32]: def get_fold_split(metadata, test_fold, val_fold=None):
        """
        Разбиение по official folds.

        Если val_fold задан:
            train = все fold'ы, кроме test_fold и val_fold
            val   = val_fold
            test  = test_fold

        Если val_fold не задан:
            train = все fold'ы, кроме test_fold
            test  = test_fold
        """
        test_df = metadata[metadata["fold"] == test_fold]

        if val_fold is not None:
            assert val_fold != test_fold
            val_df = metadata[metadata["fold"] == val_fold]
            train_folds = [f for f in range(1, 11) if f not in {test_fold, val_fold}]
        else:
            val_df = None
            train_folds = [f for f in range(1, 11) if f != test_fold]

        train_df = metadata[metadata["fold"].isin(train_folds)]

        print(
            f"test={test_fold}"
            + (f", val={val_fold}" if val_fold is not None else "")
            + f" | Train: {len(train_df)}, Val: {len(val_df) if val_df is not None else 0}"
        )
        return train_df, val_df, test_df

```

Dev-split для быстрых экспериментов

Для подбора архитектуры и отладки удобно сначала работать на одном фиксированном разбиении:

- train = folds 1-8
- val = fold 9
- test = fold 10

Финальную 10-fold оценку лучше запускать уже после того, как основная конфигурация выбрана.

```
In [33]: train_df, val_df, test_df = get_fold_split(metadata, test_fold=10, val_fold=9)

BATCH_SIZE = 32
CACHE_DIR = "./mel_cache"

train_cache_ds = UrbanSoundDataset(
    train_df, os.path.dirname(DATASET_PATH), global_label_encoder,
    augment=False,
    cache_dir=os.path.join(CACHE_DIR, "train")
)
val_cache_ds = UrbanSoundDataset(
    val_df, os.path.dirname(DATASET_PATH), global_label_encoder,
    augment=False,
    cache_dir=os.path.join(CACHE_DIR, "val")
)
test_cache_ds = UrbanSoundDataset(
    test_df, os.path.dirname(DATASET_PATH), global_label_encoder,
    augment=False,
    cache_dir=os.path.join(CACHE_DIR, "test")
)

for i in tqdm(range(len(train_cache_ds)), desc="Train cache"):
    _ = train_cache_ds[i]
for i in tqdm(range(len(val_cache_ds)), desc="Val cache"):
    _ = val_cache_ds[i]
for i in tqdm(range(len(test_cache_ds)), desc="Test cache"):
    _ = test_cache_ds[i]

train_dataset = UrbanSoundDataset(
    train_df,
    os.path.dirname(DATASET_PATH),
    global_label_encoder,
    augment=True,
    cache_dir=os.path.join(CACHE_DIR, "train")
)

val_dataset = UrbanSoundDataset(
    val_df,
    os.path.dirname(DATASET_PATH),
    global_label_encoder,
    augment=False,
    cache_dir=os.path.join(CACHE_DIR, "val")
)

test_dataset = UrbanSoundDataset(
    test_df,
    os.path.dirname(DATASET_PATH),
    global_label_encoder,
    augment=False,
    cache_dir=os.path.join(CACHE_DIR, "test")
)
```

```

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0,
    pin_memory=True,
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0,
    pin_memory=True,
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0,
    pin_memory=True,
)

sample_batch, sample_labels = next(iter(train_loader))

print(f"Batch shape: {sample_batch.shape}")
print(f"Labels shape: {sample_labels.shape}")
print(f"First labels: {sample_labels[:5]}")

```

```

test=10, val=9 | Train: 7079, Val: 816, Test: 837
Train cache: 0%|          | 0/7079 [00:00<?, ?it/s]
Val cache: 0%|          | 0/816 [00:00<?, ?it/s]
Test cache: 0%|          | 0/837 [00:00<?, ?it/s]
Batch shape: torch.Size([32, 1, 128, 173])
Labels shape: torch.Size([32])
First labels: tensor([7, 5, 2, 4, 4])

```

```

c:\Python Projects\Jupyter\.venv\Lib\site-packages\torch\utils\data\data_loader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)

```

Задание 2.2. Реализуйте вашу архитектуру (1 балл)

В этой задаче разрешены только **свёрточные архитектуры**.

Нельзя использовать:

- RNN;
- LSTM / GRU;
- Transformer.

Можно использовать: обычные CNN-блоки, batch normalization, dropout, pooling, при желании - несложные attention-модули поверх CNN, если основа модели всё

равно остаётся сверточной.

На вход модель получает мел-спектрограмму формы `(B, 1, n_mels, time)`, а на выходе должна возвращать логиты по 10 классам.

```
In [34]: class UrbanSoundCNN(nn.Module):
    def __init__(self, num_classes=10, dropout=0.5, channels=[1, 32, 64, 128, 256]):
        # TODO: Реализуйте инициализацию слоев вашей архитектуры
        # (☺~☹)

        super(UrbanSoundCNN, self).__init__()
        conv_layers = []
        in_ch = channels[0]
        for out_ch in channels[1:]:
            conv_layers += [
                nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
                nn.BatchNorm2d(out_ch),
                nn.ReLU(inplace=True),
                nn.MaxPool2d(2)
            ]
            in_ch = out_ch
        self.features = nn.Sequential(*conv_layers)

        self.classifier = nn.Sequential(
            nn.AdaptiveAvgPool2d((4, 4)),
            nn.Flatten(),
            nn.Linear(channels[-1] * 4 * 4, fc_dim),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(fc_dim, num_classes)
        )

    def forward(self, x):
        # TODO: Реализуйте forward вашей архитектуры
        # ☹(↗↖)☺
        x = self.features(x)
        x = self.classifier(x)
        return x
```

Задание 2.3. Реализуйте обучение и валидацию модели (1 балл)

Теперь нужно написать код обучения модели на dev-split.

Ожидается, что у вас будут:

- функция `train_epoch`;
- функция `validate`;
- цикл обучения с early stopping;
- сохранение лучшей модели по качеству на валидации.

Здесь нет единственно правильной реализации, но ваш код должен быть аккуратным, воспроизводимым и согласованным с поставленной задачей.

```

In [35]: # TODO (-_-):
# def train_epoch(model, loader, criterion, optimizer, device) -> (loss, acc)
# def validate(model, loader, criterion, device) -> (loss, acc)

def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    pbar = tqdm(loader, desc="Training", leave=False)
    for inputs, labels in pbar:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()
        pbar.set_postfix(loss=loss.item())
    return running_loss / total, 100. * correct / total

def validate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        pbar = tqdm(loader, desc="Validation", leave=False)
        for inputs, labels in pbar:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * inputs.size(0)
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()
    return running_loss / total, 100. * correct / total

model = UrbanSoundCNN(num_classes=10).to(device)

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"Total parameters: {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
print("\nModel:")
print(model)

```

Total parameters: 914,506
Trainable parameters: 914,506

Model:

```
UrbanSoundCNN(  
  (features): Sequential(  
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (6): ReLU(inplace=True)  
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (10): ReLU(inplace=True)  
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (12): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (13): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (14): ReLU(inplace=True)  
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (classifier): Sequential(  
    (0): AdaptiveAvgPool2d(output_size=(4, 4))  
    (1): Flatten(start_dim=1, end_dim=-1)  
    (2): Linear(in_features=4096, out_features=128, bias=True)  
    (3): ReLU(inplace=True)  
    (4): Dropout(p=0.5, inplace=False)  
    (5): Linear(in_features=128, out_features=10, bias=True)  
  )  
)
```

```
In [38]: criterion = nn.CrossEntropyLoss()  
optimizer = torch.optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)  
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(  
    optimizer, mode='min', factor=0.5, patience=5  
)  
  
NUM_EPOCHS = 15  
PATIENCE = 10  
  
history = {  
    "train_loss": [],  
    "train_acc": [],  
    "val_loss": [],  
    "val_acc": [],  
}  
  
best_val_acc = 0.0  
patience_counter = 0
```

```

# YOUR CODE ( • ε • )
# реализуйте цикл обучения на dev-split
# при улучшении val ассурасу сохраняйте веса модели

for epoch in range(1, NUM_EPOCHS + 1):
    train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer)
    val_loss, val_acc = validate(model, val_loader, criterion, device)

    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)

    scheduler.step(val_loss)

    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}%")
    print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}%")

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), "best_cnn_model.pth")
        print(f"--> New best model saved (val_acc = {val_acc:.2f}%)")
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= PATIENCE:
            print(f"Early stopping after {epoch} epochs")
            break

print(f"\nTraining finished. Best validation accuracy: {best_val_acc:.2f}%")

```

```

Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 1.1322 | Train Acc: 59.53%
Val Loss: 1.2389 | Val Acc: 62.13%
--> New best model saved (val_acc = 62.13%)
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.9610 | Train Acc: 65.80%
Val Loss: 0.9448 | Val Acc: 68.14%
--> New best model saved (val_acc = 68.14%)
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.8197 | Train Acc: 72.00%
Val Loss: 0.8366 | Val Acc: 71.94%
--> New best model saved (val_acc = 71.94%)
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.7332 | Train Acc: 74.64%
Val Loss: 0.8793 | Val Acc: 71.08%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.6499 | Train Acc: 78.26%
Val Loss: 0.8964 | Val Acc: 74.14%
--> New best model saved (val_acc = 74.14%)
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.5685 | Train Acc: 80.70%
Val Loss: 0.7562 | Val Acc: 80.02%
--> New best model saved (val_acc = 80.02%)

```

```

Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.5308 | Train Acc: 82.55%
Val Loss: 1.1302 | Val Acc: 68.87%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.4952 | Train Acc: 83.27%
Val Loss: 0.9858 | Val Acc: 78.31%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.4401 | Train Acc: 84.86%
Val Loss: 0.9368 | Val Acc: 76.59%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.4138 | Train Acc: 86.07%
Val Loss: 0.7455 | Val Acc: 79.53%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.3768 | Train Acc: 87.37%
Val Loss: 0.7160 | Val Acc: 83.21%
--> New best model saved (val_acc = 83.21%)
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.3455 | Train Acc: 88.35%
Val Loss: 1.5377 | Val Acc: 68.50%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.3452 | Train Acc: 88.42%
Val Loss: 0.7357 | Val Acc: 82.11%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.3277 | Train Acc: 89.50%
Val Loss: 1.2954 | Val Acc: 70.83%
Training:  0%|          | 0/222 [00:00<?, ?it/s]
Validation: 0%|          | 0/26 [00:00<?, ?it/s]
Train Loss: 0.3138 | Train Acc: 89.56%
Val Loss: 0.8096 | Val Acc: 80.39%

```

Training finished. Best validation accuracy: 83.21%

После обучения оцените модель на test-части dev-split. Затем постройте confusion matrix и посмотрите, какие классы модель путает чаще всего.

```

In [41]: model.load_state_dict(torch.load("best_cnn_model.pth", map_location=device))
         model.eval()

         all_preds = []
         all_labels = []

         with torch.no_grad():
             for inputs, labels in tqdm(test_loader, desc="Testing"):
                 inputs = inputs.to(device)
                 outputs = model(inputs)
                 preds = outputs.argmax(dim=1)

                 all_preds.extend(preds.cpu().numpy())
                 all_labels.extend(labels.numpy())

```

```
test_acc = np.mean(np.array(all_preds) == np.array(all_labels))
print(f"Test accuracy on dev-split: {test_acc * 100:.2f}%")
```

c:\Python Projects\Jupyter\.venv\Lib\site-packages\torch\utils\data\dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.

```
super().__init__(loader)
```

Testing: 0%| | 0/27 [00:00<?, ?it/s]

Test accuracy on dev-split: 75.39%

In [42]: `import seaborn as sns`

```
cm = confusion_matrix(all_labels, all_preds)
class_names = global_label_encoder.classes_
```

```
plt.figure(figsize=(12, 10))
```

```
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names,
    cbar_kws={"label": "Count"},
)
```

```
plt.title("Confusion Matrix")
```

```
plt.xlabel("Predicted label")
```

```
plt.ylabel("True label")
```

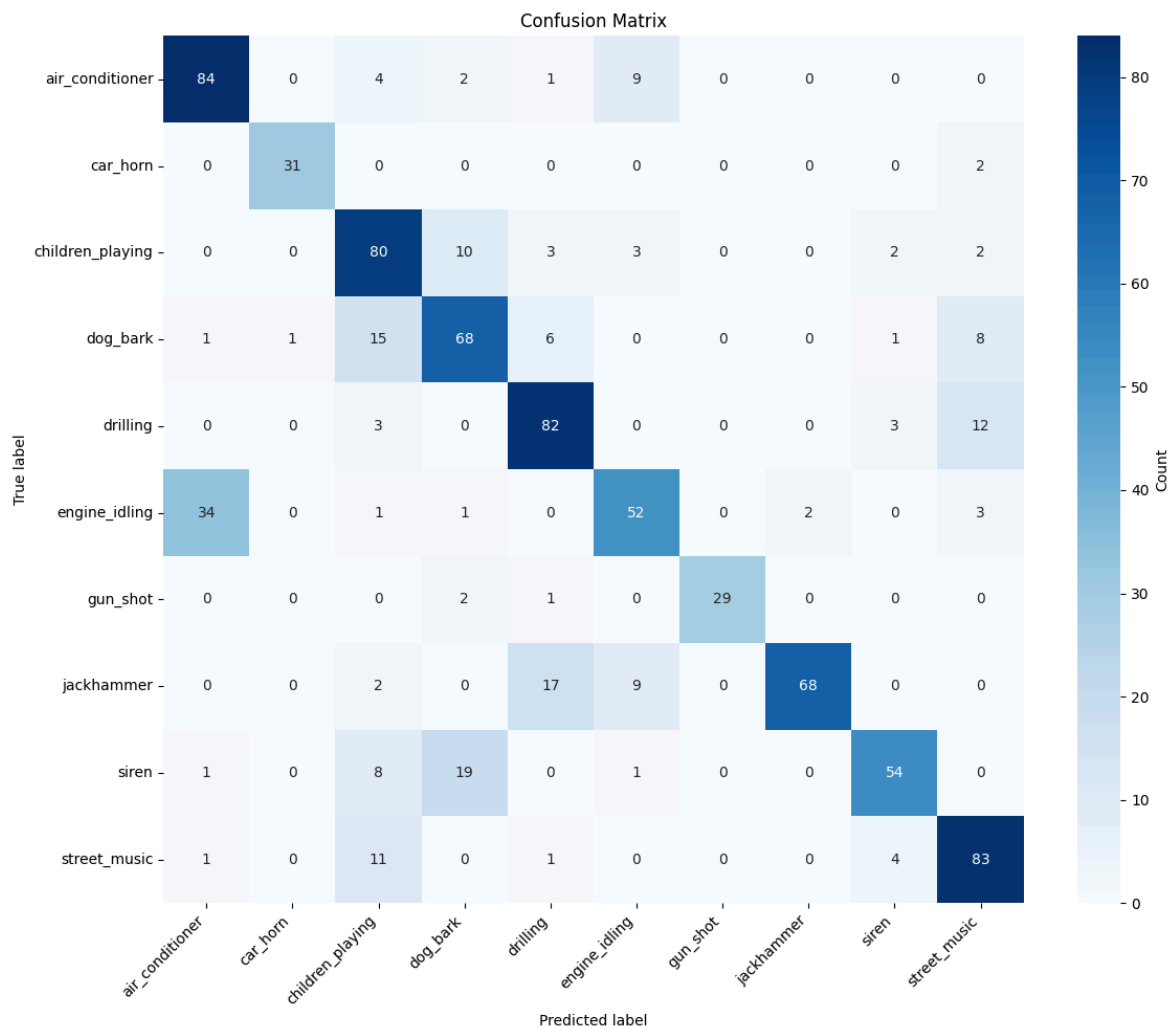
```
plt.xticks(rotation=45, ha="right")
```

```
plt.yticks(rotation=0)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print(classification_report(all_labels, all_preds, target_names=class_names, dig
```



	precision	recall	f1-score	support
air_conditioner	0.6942	0.8400	0.7602	100
car_horn	0.9688	0.9394	0.9538	33
children_playing	0.6452	0.8000	0.7143	100
dog_bark	0.6667	0.6800	0.6733	100
drilling	0.7387	0.8200	0.7773	100
engine_idling	0.7027	0.5591	0.6228	93
gun_shot	1.0000	0.9062	0.9508	32
jackhammer	0.9714	0.7083	0.8193	96
siren	0.8438	0.6506	0.7347	83
street_music	0.7545	0.8300	0.7905	100
accuracy			0.7539	837
macro avg	0.7986	0.7734	0.7797	837
weighted avg	0.7677	0.7539	0.7539	837

Задание 2.4. Финальная 10-fold оценка (2,5 балла)

После того как вы подобрали рабочую конфигурацию на dev-split, проведите один финальный прогон 10-fold cross-validation.

Для каждого `test_fold`:

- `val_fold` берётся как следующий fold по кругу;
- все остальные fold'ы идут в train.

Итоговое качество считается по средней accuracy по всем 10 fold'ам.

```
In [43]: def evaluate_10fold(
    model_class,
    model_kwargs,
    train_fn,
    metadata,
    dataset_path,
    label_encoder,
    device
):
    """
    10-fold CV no folds UrbanSound8K
    """
    accuracies = []

    for test_fold in range(1, 11):
        val_fold = 1 if test_fold == 10 else test_fold + 1
        train_folds = [f for f in range(1, 11) if f not in {test_fold, val_fold}]

        print(f"\n{'=' * 50}")
        print(f"Fold {test_fold}/10 (val={val_fold}, train={train_folds})")

        train_df = metadata[metadata["fold"].isin(train_folds)]
        val_df = metadata[metadata["fold"] == val_fold]
        test_df = metadata[metadata["fold"] == test_fold]

        train_ds = UrbanSoundDataset(train_df, dataset_path, label_encoder, augmen
        val_ds = UrbanSoundDataset(val_df, dataset_path, label_encoder, augment=
        test_ds = UrbanSoundDataset(test_df, dataset_path, label_encoder, augmen

        loader_kwargs = dict(num_workers=0, pin_memory=False)
        train_loader = DataLoader(train_ds, batch_size=64, shuffle=True, **loade
        val_loader = DataLoader(val_ds, batch_size=64, shuffle=False, **loader_k
        test_loader = DataLoader(test_ds, batch_size=64, shuffle=False, **loader

        model = model_class(**model_kwargs).to(device)
        model = train_fn(model, train_loader, val_loader, device)

        model.eval()
        all_preds, all_labels = [], []
        with torch.no_grad():
            for inputs, labels in test_loader:
                inputs = inputs.to(device)
                preds = model(inputs).argmax(1)
                all_preds.extend(preds.cpu().numpy())
                all_labels.extend(labels.numpy())

        acc = np.mean(np.array(all_preds) == np.array(all_labels))
        accuracies.append(acc)
        print(f"Fold {test_fold} accuracy: {acc * 100:.2f}%")

    del model

    mean_acc = np.mean(accuracies)
    std_acc = np.std(accuracies)
    print(f"\n{'=' * 50}")
    print(f"10-fold CV: {mean_acc * 100:.2f}% ± {std_acc * 100:.2f}%")
    return accuracies
```



```
In [44]: def my_train_fn(model, train_loader, val_loader, device):
        """
        Оберните свой training loop в функцию.
        Она должна вернуть обученную модель.
        """

        criterion = nn.CrossEntropyLoss()
        optimizer = torch.optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
        scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
            optimizer, mode='min', factor=0.5, patience=5
        )

        best_val_acc = 0.0
        patience_counter = 0
        patience = 10
        num_epochs = 15

        for epoch in range(1, num_epochs + 1):
            train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
            val_loss, val_acc = validate(model, val_loader, criterion, device)

            scheduler.step(val_loss)

            print(f"Epoch {epoch:2d} | Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                patience_counter = 0
            else:
                patience_counter += 1
                if patience_counter >= patience:
                    print(f"Early stopping at epoch {epoch}")
                    break

        return model
```

```
In [45]: accuracies = evaluate_10fold(
    model_class=UrbanSoundCNN,
    model_kwargs={"num_classes": 10},
    train_fn=my_train_fn,
    metadata=metadata,
    dataset_path=os.path.dirname(DATASET_PATH),
    label_encoder=global_label_encoder,
    device=device,
)

=====
Fold 1/10 (val=2, train=[3, 4, 5, 6, 7, 8, 9, 10])
Training: 0%|          | 0/109 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.7509 | Train Acc: 35.33% | Val Loss: 1.5459 | Val Acc: 37.84%
Training: 0%|          | 0/109 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.3133 | Train Acc: 52.72% | Val Loss: 1.2582 | Val Acc: 56.08%
Training: 0%|          | 0/109 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
```

Epoch 3 | Train Loss: 1.1108 | Train Acc: 59.39% | Val Loss: 1.1416 | Val Acc: 53.49%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.9648 | Train Acc: 66.15% | Val Loss: 1.0987 | Val Acc: 59.12%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.8615 | Train Acc: 69.66% | Val Loss: 1.5940 | Val Acc: 48.09%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.7776 | Train Acc: 73.75% | Val Loss: 1.3045 | Val Acc: 59.57%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.6929 | Train Acc: 76.46% | Val Loss: 1.1389 | Val Acc: 59.35%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.6733 | Train Acc: 76.66% | Val Loss: 1.1891 | Val Acc: 61.37%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.6166 | Train Acc: 79.82% | Val Loss: 0.9569 | Val Acc: 65.32%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.5337 | Train Acc: 81.72% | Val Loss: 1.4025 | Val Acc: 64.30%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.5135 | Train Acc: 82.51% | Val Loss: 0.9089 | Val Acc: 69.26%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.5004 | Train Acc: 83.35% | Val Loss: 2.2908 | Val Acc: 66.33%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4784 | Train Acc: 83.53% | Val Loss: 0.9758 | Val Acc: 71.17%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4600 | Train Acc: 84.91% | Val Loss: 1.0615 | Val Acc: 68.02%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.4142 | Train Acc: 86.00% | Val Loss: 0.9442 | Val Acc: 69.26%
Fold 1 accuracy: 69.99%

=====
Fold 2/10 (val=3, train=[1, 4, 5, 6, 7, 8, 9, 10])
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.7542 | Train Acc: 36.84% | Val Loss: 1.4928 | Val Acc: 43.14%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 2 | Train Loss: 1.2367 | Train Acc: 56.42% | Val Loss: 1.3023 | Val Acc: 47.78%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 1.0520 | Train Acc: 64.24% | Val Loss: 1.2450 | Val Acc: 53.19%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.9616 | Train Acc: 67.55% | Val Loss: 1.5252 | Val Acc: 45.30%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.8461 | Train Acc: 71.24% | Val Loss: 1.0964 | Val Acc: 63.78%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.7750 | Train Acc: 73.98% | Val Loss: 1.5305 | Val Acc: 49.19%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.7523 | Train Acc: 75.10% | Val Loss: 1.4826 | Val Acc: 59.35%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.6844 | Train Acc: 77.29% | Val Loss: 1.2787 | Val Acc: 63.57%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.6580 | Train Acc: 77.77% | Val Loss: 1.4510 | Val Acc: 62.16%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.5796 | Train Acc: 80.73% | Val Loss: 1.3424 | Val Acc: 63.89%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.5748 | Train Acc: 80.95% | Val Loss: 1.5178 | Val Acc: 64.65%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.4652 | Train Acc: 84.59% | Val Loss: 1.5170 | Val Acc: 64.00%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4388 | Train Acc: 85.68% | Val Loss: 2.0105 | Val Acc: 62.81%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4482 | Train Acc: 85.07% | Val Loss: 1.4122 | Val Acc: 63.78%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.3903 | Train Acc: 87.15% | Val Loss: 1.4742 | Val Acc: 61.95%
Fold 2 accuracy: 69.37%

=====
Fold 3/10 (val=4, train=[1, 2, 5, 6, 7, 8, 9, 10])
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]

Epoch 1 | Train Loss: 1.5964 | Train Acc: 41.43% | Val Loss: 1.7452 | Val Acc: 42.73%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.1525 | Train Acc: 58.84% | Val Loss: 1.5252 | Val Acc: 52.93%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 0.9500 | Train Acc: 67.49% | Val Loss: 1.2081 | Val Acc: 58.08%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.8392 | Train Acc: 71.51% | Val Loss: 1.2119 | Val Acc: 59.60%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.7068 | Train Acc: 76.97% | Val Loss: 2.1211 | Val Acc: 49.39%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.7111 | Train Acc: 77.13% | Val Loss: 0.9293 | Val Acc: 68.48%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.6209 | Train Acc: 79.82% | Val Loss: 1.3743 | Val Acc: 66.16%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.5575 | Train Acc: 81.66% | Val Loss: 1.1250 | Val Acc: 63.54%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.5247 | Train Acc: 82.57% | Val Loss: 1.1174 | Val Acc: 70.71%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.4919 | Train Acc: 83.61% | Val Loss: 1.6094 | Val Acc: 63.33%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.4634 | Train Acc: 85.10% | Val Loss: 1.3100 | Val Acc: 63.74%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.4491 | Train Acc: 85.73% | Val Loss: 0.9146 | Val Acc: 70.51%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4235 | Train Acc: 86.42% | Val Loss: 1.6786 | Val Acc: 62.93%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4085 | Train Acc: 87.08% | Val Loss: 1.6559 | Val Acc: 63.54%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/16 [00:00<?, ?it/s]

Epoch 15 | Train Loss: 0.3900 | Train Acc: 87.09% | Val Loss: 1.2742 | Val Acc: 65.86%

Fold 3 accuracy: 65.95%

=====
Fold 4/10 (val=5, train=[1, 2, 3, 6, 7, 8, 9, 10])

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 1 | Train Loss: 1.6653 | Train Acc: 38.51% | Val Loss: 2.1397 | Val Acc: 35.26%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 2 | Train Loss: 1.1961 | Train Acc: 57.35% | Val Loss: 1.5415 | Val Acc: 48.61%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 3 | Train Loss: 0.9978 | Train Acc: 65.87% | Val Loss: 0.8693 | Val Acc: 68.80%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 4 | Train Loss: 0.8754 | Train Acc: 70.20% | Val Loss: 1.1518 | Val Acc: 56.62%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 5 | Train Loss: 0.7984 | Train Acc: 72.73% | Val Loss: 0.8993 | Val Acc: 65.06%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 6 | Train Loss: 0.7599 | Train Acc: 74.60% | Val Loss: 1.2539 | Val Acc: 65.60%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 7 | Train Loss: 0.6906 | Train Acc: 77.28% | Val Loss: 0.7298 | Val Acc: 76.39%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 8 | Train Loss: 0.6272 | Train Acc: 79.24% | Val Loss: 0.9207 | Val Acc: 71.26%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 9 | Train Loss: 0.5971 | Train Acc: 79.74% | Val Loss: 0.7744 | Val Acc: 76.60%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 10 | Train Loss: 0.5537 | Train Acc: 81.62% | Val Loss: 1.2332 | Val Acc: 68.48%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 11 | Train Loss: 0.5058 | Train Acc: 83.31% | Val Loss: 0.8697 | Val Acc: 67.41%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 12 | Train Loss: 0.5004 | Train Acc: 83.41% | Val Loss: 1.1150 | Val Acc: 69.12%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 13 | Train Loss: 0.4296 | Train Acc: 85.62% | Val Loss: 0.7938 | Val Acc: 76.71%

Training: 0%| | 0/107 [00:00<?, ?it/s]

Validation: 0%| | 0/15 [00:00<?, ?it/s]

Epoch 14 | Train Loss: 0.3478 | Train Acc: 88.33% | Val Loss: 0.9335 | Val Acc: 72.54%
Training: 0%| | 0/107 [00:00<?, ?it/s]
Validation: 0%| | 0/15 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.3288 | Train Acc: 89.01% | Val Loss: 0.5193 | Val Acc: 85.04%
Fold 4 accuracy: 74.14%

```
=====
Fold 5/10 (val=6, train=[1, 2, 3, 4, 7, 8, 9, 10])
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.7324 | Train Acc: 35.38% | Val Loss: 1.4776 | Val Acc: 44.11%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.3121 | Train Acc: 51.97% | Val Loss: 1.3313 | Val Acc: 56.14%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 1.1312 | Train Acc: 58.51% | Val Loss: 1.3311 | Val Acc: 54.68%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.9825 | Train Acc: 64.74% | Val Loss: 1.0723 | Val Acc: 63.18%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.8799 | Train Acc: 69.01% | Val Loss: 1.3651 | Val Acc: 60.63%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.8245 | Train Acc: 71.20% | Val Loss: 1.2747 | Val Acc: 60.27%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.7368 | Train Acc: 74.70% | Val Loss: 1.0312 | Val Acc: 72.30%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.6898 | Train Acc: 76.19% | Val Loss: 0.8666 | Val Acc: 71.93%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.6465 | Train Acc: 78.24% | Val Loss: 1.2030 | Val Acc: 67.31%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.5791 | Train Acc: 80.44% | Val Loss: 1.2352 | Val Acc: 68.04%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.5409 | Train Acc: 81.84% | Val Loss: 1.1000 | Val Acc: 68.41%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.5106 | Train Acc: 83.02% | Val Loss: 1.2925 | Val Acc: 68.41%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
```

Epoch 13 | Train Loss: 0.4682 | Train Acc: 84.51% | Val Loss: 1.2759 | Val Acc: 69.62%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4591 | Train Acc: 84.34% | Val Loss: 1.1001 | Val Acc: 71.45%
Training: 0%| | 0/109 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.3616 | Train Acc: 88.03% | Val Loss: 0.9605 | Val Acc: 75.46%
Fold 5 accuracy: 85.47%

=====
Fold 6/10 (val=7, train=[1, 2, 3, 4, 5, 8, 9, 10])
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.6654 | Train Acc: 38.33% | Val Loss: 1.4547 | Val Acc: 40.10%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.2273 | Train Acc: 55.86% | Val Loss: 1.4205 | Val Acc: 56.80%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 1.0985 | Train Acc: 60.02% | Val Loss: 1.1728 | Val Acc: 59.90%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.9507 | Train Acc: 67.03% | Val Loss: 1.3664 | Val Acc: 56.92%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.8725 | Train Acc: 69.98% | Val Loss: 1.3492 | Val Acc: 59.31%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.7908 | Train Acc: 73.45% | Val Loss: 1.6285 | Val Acc: 57.76%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.7403 | Train Acc: 75.05% | Val Loss: 0.9835 | Val Acc: 66.11%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.6796 | Train Acc: 77.27% | Val Loss: 1.1675 | Val Acc: 62.89%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.6504 | Train Acc: 78.57% | Val Loss: 0.9783 | Val Acc: 68.74%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.5868 | Train Acc: 80.21% | Val Loss: 1.1773 | Val Acc: 63.60%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.5520 | Train Acc: 81.16% | Val Loss: 2.1340 | Val Acc: 54.77%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]

Epoch 12 | Train Loss: 0.5180 | Train Acc: 82.31% | Val Loss: 1.0970 | Val Acc: 66.95%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4914 | Train Acc: 83.79% | Val Loss: 1.3907 | Val Acc: 68.50%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4586 | Train Acc: 85.07% | Val Loss: 1.4902 | Val Acc: 61.34%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.4287 | Train Acc: 85.31% | Val Loss: 1.3969 | Val Acc: 68.38%
Fold 6 accuracy: 73.15%

=====
Fold 7/10 (val=8, train=[1, 2, 3, 4, 5, 6, 9, 10])
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.7428 | Train Acc: 35.09% | Val Loss: 1.5160 | Val Acc: 44.17%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.2736 | Train Acc: 52.98% | Val Loss: 1.2112 | Val Acc: 54.71%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 1.0562 | Train Acc: 61.64% | Val Loss: 1.2925 | Val Acc: 59.06%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.9183 | Train Acc: 66.89% | Val Loss: 1.2278 | Val Acc: 62.90%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.8606 | Train Acc: 69.96% | Val Loss: 1.7562 | Val Acc: 55.58%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.7863 | Train Acc: 72.87% | Val Loss: 1.3257 | Val Acc: 65.76%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.7555 | Train Acc: 73.60% | Val Loss: 1.4994 | Val Acc: 59.43%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.6421 | Train Acc: 77.72% | Val Loss: 1.6032 | Val Acc: 63.90%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.5445 | Train Acc: 81.79% | Val Loss: 1.2605 | Val Acc: 70.72%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.4785 | Train Acc: 84.14% | Val Loss: 1.7706 | Val Acc: 61.66%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 11 | Train Loss: 0.4836 | Train Acc: 84.02% | Val Loss: 1.0291 | Val Acc: 69.73%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.4457 | Train Acc: 84.58% | Val Loss: 2.0640 | Val Acc: 61.79%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4243 | Train Acc: 85.68% | Val Loss: 1.4447 | Val Acc: 69.11%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.3990 | Train Acc: 86.88% | Val Loss: 1.4407 | Val Acc: 73.45%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.3814 | Train Acc: 86.95% | Val Loss: 1.6988 | Val Acc: 67.74%
Fold 7 accuracy: 73.39%

=====
Fold 8/10 (val=9, train=[1, 2, 3, 4, 5, 6, 7, 10])
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.6498 | Train Acc: 39.21% | Val Loss: 1.7746 | Val Acc: 39.83%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.1693 | Train Acc: 58.68% | Val Loss: 1.0181 | Val Acc: 68.75%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 0.9966 | Train Acc: 64.71% | Val Loss: 0.9115 | Val Acc: 70.59%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.8606 | Train Acc: 70.38% | Val Loss: 1.0771 | Val Acc: 63.60%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.7762 | Train Acc: 73.78% | Val Loss: 0.8622 | Val Acc: 74.14%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.7263 | Train Acc: 76.16% | Val Loss: 1.0669 | Val Acc: 69.12%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.7250 | Train Acc: 76.23% | Val Loss: 0.7728 | Val Acc: 74.75%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.6375 | Train Acc: 78.79% | Val Loss: 1.1272 | Val Acc: 70.96%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.5700 | Train Acc: 81.35% | Val Loss: 0.6962 | Val Acc: 80.51%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]

Epoch 10 | Train Loss: 0.5076 | Train Acc: 82.97% | Val Loss: 0.8291 | Val Acc: 83.82%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.5161 | Train Acc: 82.95% | Val Loss: 1.0309 | Val Acc: 69.24%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.4567 | Train Acc: 85.43% | Val Loss: 0.8520 | Val Acc: 80.27%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4287 | Train Acc: 86.37% | Val Loss: 0.9282 | Val Acc: 75.00%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4087 | Train Acc: 86.81% | Val Loss: 0.7627 | Val Acc: 78.31%
Training: 0%| | 0/112 [00:00<?, ?it/s]
Validation: 0%| | 0/13 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.3795 | Train Acc: 87.50% | Val Loss: 0.8108 | Val Acc: 79.53%
Fold 8 accuracy: 65.76%

=====
Fold 9/10 (val=10, train=[1, 2, 3, 4, 5, 6, 7, 8])
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.6638 | Train Acc: 38.62% | Val Loss: 1.2290 | Val Acc: 55.32%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.2538 | Train Acc: 54.46% | Val Loss: 1.1425 | Val Acc: 55.68%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 1.0494 | Train Acc: 62.06% | Val Loss: 1.0699 | Val Acc: 61.65%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.9189 | Train Acc: 67.58% | Val Loss: 1.1313 | Val Acc: 62.25%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.8339 | Train Acc: 71.10% | Val Loss: 1.1949 | Val Acc: 70.37%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.7495 | Train Acc: 75.32% | Val Loss: 0.8714 | Val Acc: 73.36%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.6881 | Train Acc: 76.47% | Val Loss: 0.7605 | Val Acc: 75.27%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.6510 | Train Acc: 78.30% | Val Loss: 0.8817 | Val Acc: 68.70%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]

Epoch 9 | Train Loss: 0.5836 | Train Acc: 80.72% | Val Loss: 0.9653 | Val Acc: 68.22%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.5751 | Train Acc: 80.72% | Val Loss: 0.9436 | Val Acc: 73.84%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.4939 | Train Acc: 83.50% | Val Loss: 0.9492 | Val Acc: 75.51%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.4694 | Train Acc: 84.60% | Val Loss: 0.6931 | Val Acc: 77.66%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4428 | Train Acc: 84.98% | Val Loss: 0.7443 | Val Acc: 76.22%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4251 | Train Acc: 85.87% | Val Loss: 0.6817 | Val Acc: 79.45%
Training: 0%| | 0/111 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.4087 | Train Acc: 86.62% | Val Loss: 0.8072 | Val Acc: 77.42%
Fold 9 accuracy: 79.17%

=====
Fold 10/10 (val=1, train=[2, 3, 4, 5, 6, 7, 8, 9])
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 1.7319 | Train Acc: 36.66% | Val Loss: 1.5233 | Val Acc: 44.90%
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 1.2756 | Train Acc: 54.09% | Val Loss: 1.1715 | Val Acc: 55.33%
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 1.0795 | Train Acc: 61.05% | Val Loss: 1.6482 | Val Acc: 48.22%
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.9755 | Train Acc: 65.99% | Val Loss: 1.0895 | Val Acc: 61.74%
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.8683 | Train Acc: 70.05% | Val Loss: 1.9223 | Val Acc: 43.64%
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.8245 | Train Acc: 71.55% | Val Loss: 1.7208 | Val Acc: 55.90%
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.7126 | Train Acc: 75.56% | Val Loss: 0.9459 | Val Acc: 65.98%
Training: 0%| | 0/110 [00:00<?, ?it/s]
Validation: 0%| | 0/14 [00:00<?, ?it/s]

```

Epoch 8 | Train Loss: 0.6821 | Train Acc: 75.92% | Val Loss: 1.8680 | Val Acc: 54.07%
Training: 0%|          | 0/110 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.6274 | Train Acc: 78.70% | Val Loss: 1.4013 | Val Acc: 64.49%
Training: 0%|          | 0/110 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.5928 | Train Acc: 79.61% | Val Loss: 0.9435 | Val Acc: 72.05%
Training: 0%|          | 0/110 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 11 | Train Loss: 0.5615 | Train Acc: 80.90% | Val Loss: 1.7782 | Val Acc: 58.76%
Training: 0%|          | 0/110 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 12 | Train Loss: 0.5089 | Train Acc: 83.22% | Val Loss: 0.9890 | Val Acc: 68.61%
Training: 0%|          | 0/110 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 13 | Train Loss: 0.4725 | Train Acc: 84.15% | Val Loss: 1.2527 | Val Acc: 71.94%
Training: 0%|          | 0/110 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 14 | Train Loss: 0.4598 | Train Acc: 84.45% | Val Loss: 1.5950 | Val Acc: 63.34%
Training: 0%|          | 0/110 [00:00<?, ?it/s]
Validation: 0%|          | 0/14 [00:00<?, ?it/s]
Epoch 15 | Train Loss: 0.4522 | Train Acc: 84.82% | Val Loss: 1.8611 | Val Acc: 56.59%
Fold 10 accuracy: 64.04%

```

```

=====
10-fold CV: 72.04% ± 6.25%

```

```

In [46]: mean_acc = np.mean(accuracies)
         std_acc = np.std(accuracies)

         for i, acc in enumerate(accuracies, 1):
             print(f"Fold {i}: {acc * 100:.2f}%")

         print(f"\n10-fold CV: {mean_acc * 100:.2f}% ± {std_acc * 100:.2f}%")

```

```

Fold 1: 69.99%
Fold 2: 69.37%
Fold 3: 65.95%
Fold 4: 74.14%
Fold 5: 85.47%
Fold 6: 73.15%
Fold 7: 73.39%
Fold 8: 65.76%
Fold 9: 79.17%
Fold 10: 64.04%

```

```

10-fold CV: 72.04% ± 6.25%

```

Баллы за пункт 2.4 начисляются в зависимости от лучшего выполненного порога по средней accuracy в 10-fold CV: чем выше итоговое качество, тем больше доля из 2,5 балла за эту часть.

```
In [47]: score_2_4 = 0.0

if mean_acc >= 0.64:
    score_2_4 = 0.5 * 2.5
if mean_acc >= 0.69:
    score_2_4 = 0.6 * 2.5
if mean_acc >= 0.74:
    score_2_4 = 0.7 * 2.5
if mean_acc >= 0.79:
    score_2_4 = 0.8 * 2.5
if mean_acc >= 0.84:
    score_2_4 = 0.9 * 2.5
if mean_acc >= 0.88:
    score_2_4 = 1.0 * 2.5

print(f"Mean CV accuracy: {mean_acc:.4f}")
print(f"Баллы за пункт 2.4: {score_2_4:.2f} / 2.5")
```

Mean CV accuracy: 0.7204

Баллы за пункт 2.4: 1.50 / 2.5

Задание 2.5. Краткий анализ (0,5 балла)

В конце прокомментируйте свою работу:

1. Какую архитектуру вы выбрали и почему.
 2. Какие аугментации оказались полезными, а какие нет.
 3. Что сильнее влияло на качество: сама CNN, preprocessing или параметры обучения.
 4. Какие классы модель путала чаще всего и почему это выглядит правдоподобно.
 5. Насколько различались результаты между fold'ами.
-
1. Очень усердно общался с гпт, чтобы понять, какие варианты можно реализовать + какие лучше подходили. В итоге остановился на этом, потому что остальные варианты давали какое-то неадекватное время обучения. Dropout использовал для избежания переобучения, BatchNorm для более стабильной сходимости, использовал три слоя как некий компромисс между точностью и скоростью.
 2. Frequency masking помог улучшить точность, полагаю, что модель научилась лучше обобщать за счет него.
 3. Параметры обучения не особо сильно спасали, но какие-то трешхолды за счет них получалось пробить. Остальное не особо перебирал
 4. Игра ребенка и лай собаки), без понятия, что с ней не так, но в принципе сходство веселое. Звук двигателя и кондиционер, вполне объяснимо и интуитивно. Отбойный молоток и дрель, тоже вполне логично и понятно. Сирена и лай собаки тоже что-то из жанра фантастики.
 5. Довольно сильно, там разница могла быть +-10%, полагаю, что это связано с неравномерным распределением звуков внутри самих фолдов, поэтому куда-то попадало больше проблемных аудио, куда-то меньше и получалось либо хуже,

либо лучше на них обучиться + может у каких-то аудио качество получше/
похуже.